

Introducción a GIT

José Vicente Berná Martínez

Profesor Titular de Universidad

Dpto. Tecnología Informática y Computación



Universitat d'Alacant
Universidad de Alicante



Grau en Enginyeria Multimèdia
Grado en Ingeniería Multimedia

Índice de contenidos

1.	Preambulo	3
2.	Instalación	3
3.	Montar repositorio y estructura	5
3.1.	Configuración de variables globales	5
3.2.	Cómo trabajar con Git: Gitflow Workflow	5
3.3.	Crear un repositorio.....	6
4.	Trabajo en grupo	7
4.1.	Sincronizar ramas	12
4.2.	Creación rama “develop”	12
4.3.	Sincronización de la rama develop	14
4.4.	Trabajo paralelo en ramas de features	15
4.5.	Fusión de la rama de backend	17
4.6.	Fusión de la rama de frontend y resolución de conflictos	17
4.6.1.	Descarte de errores comunes.....	17
4.6.2.	Sincronización previa obligatoria	17
4.6.3.	Fusión con conflicto	18
4.6.4.	Resolución manual del conflicto en VS Code	18
4.6.5.	Finalización de la fusión.....	19
5.	Trabajo con Visual Studio Code	19
5.1.	Crear una rama	19
5.2.	Cambiar de rama.....	20
5.3.	Guardar cambios	20
5.4.	Subir cambios al servidor	20
5.5.	Descargar cambios del servidor	20
5.6.	Actualizar el mapa del repositorio	20
6.	Plugin Gitgraph	21
7.	Ver y borrar ramas	23
7.1.	Sincronización y comprobación inicial	23
7.2.	Borrar la rama local	23
7.3.	Borrar la rama en el servidor remoto (GitHub).....	24

7.4.	Limpieza de referencias locales	24
7.5.	Paso a producción.....	25
8.	Ejemplo práctico con características	26
8.1.	Creación y salto a la nueva funcionalidad.....	26
8.2.	Modificación de archivos e indexación	26
8.3.	Fusión en la rama de integración local	26
8.4.	Sincronización remota de todas las ramas afectadas	27
9.	Estados de ramas y ejemplos	27
9.1.	Ramas alineadas.....	27
9.2.	Creación de la rama otra y primer desvío	27
9.3.	Integración en desarrollo.....	28
9.4.	Despliegue a producción.....	28
9.5.	Limpieza de la rama temporal	29
9.6.	Ramas paralelas, integración parcial y resolución de conflictos	30
9.6.1.	Configuración de la primera característica	30
9.6.2.	Trabajo en paralelo con la segunda característica	30
9.6.3.	Fusión y limpieza de la segunda característica	31
9.6.4.	Despliegue anticipado a producción	31
9.6.5.	Continuación del desarrollo en paralelo (prueba3).....	32
9.6.6.	Finalización y subida de la rama rezagada	33
9.6.7.	Fusión de la rama conflictiva en develop y resolución en VS Code	33
9.6.8.	Sincronizar ramas	34
9.6.9.	Limpieza de ramas de características	35
9.6.10.	Despliegue a producción	36

1. Preambulo

El objetivo de este manual es facilitar y refrescar los conocimientos mínimos sobre el uso de repositorios de código basado en GIT, lo cual será imprescindible para mantener una buena gestión de los proyectos en la asignatura. Si el alumnado ya posee agilidad en el manejo de repositorios, puede realizar una revisión rápida de los conceptos. Este manual no es un texto exhaustivo para el uso de git, para mayor profundidad en los conocimientos se invita al lector a consultar la sección Learn (<https://git-scm.com/learn>) de git.

2. Instalación

Accede a la página web oficial de GIT (<https://git-scm.com>), luego pulsa en el botón que dice "Install" para acceder a la página de descargas (la imagen de la página web puede ser distinta cuando accedas).

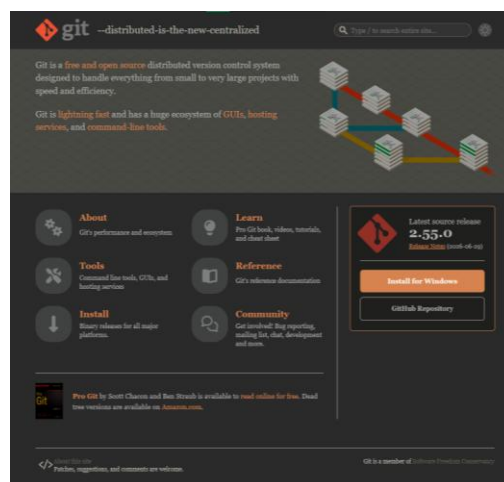


Figura 1. Captura de la web de Git para descargar.

A continuación, descarga la versión correspondiente a tu sistema operativo (Windows, macOS o Linux):

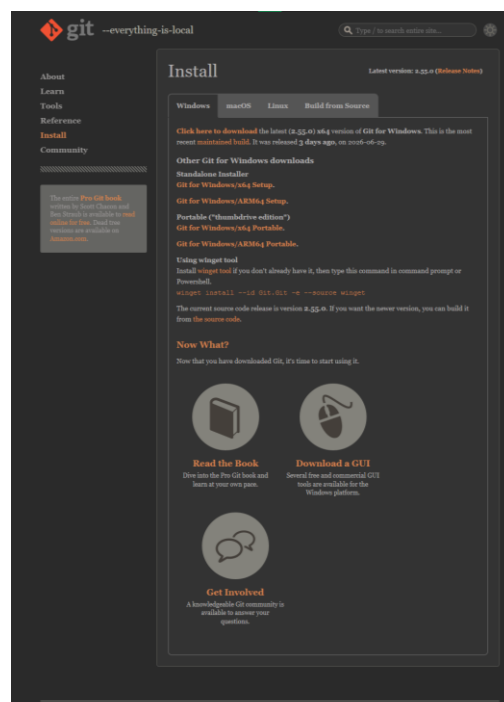


Figura 2. Captura de la web de Git para selección de versión.

Ejecutamos el instalador que hemos descargado y dejamos todo por defecto, solo eligiendo nuestro editor de código preferido (el que tú vayas a utilizar) cuando nos lo piden:

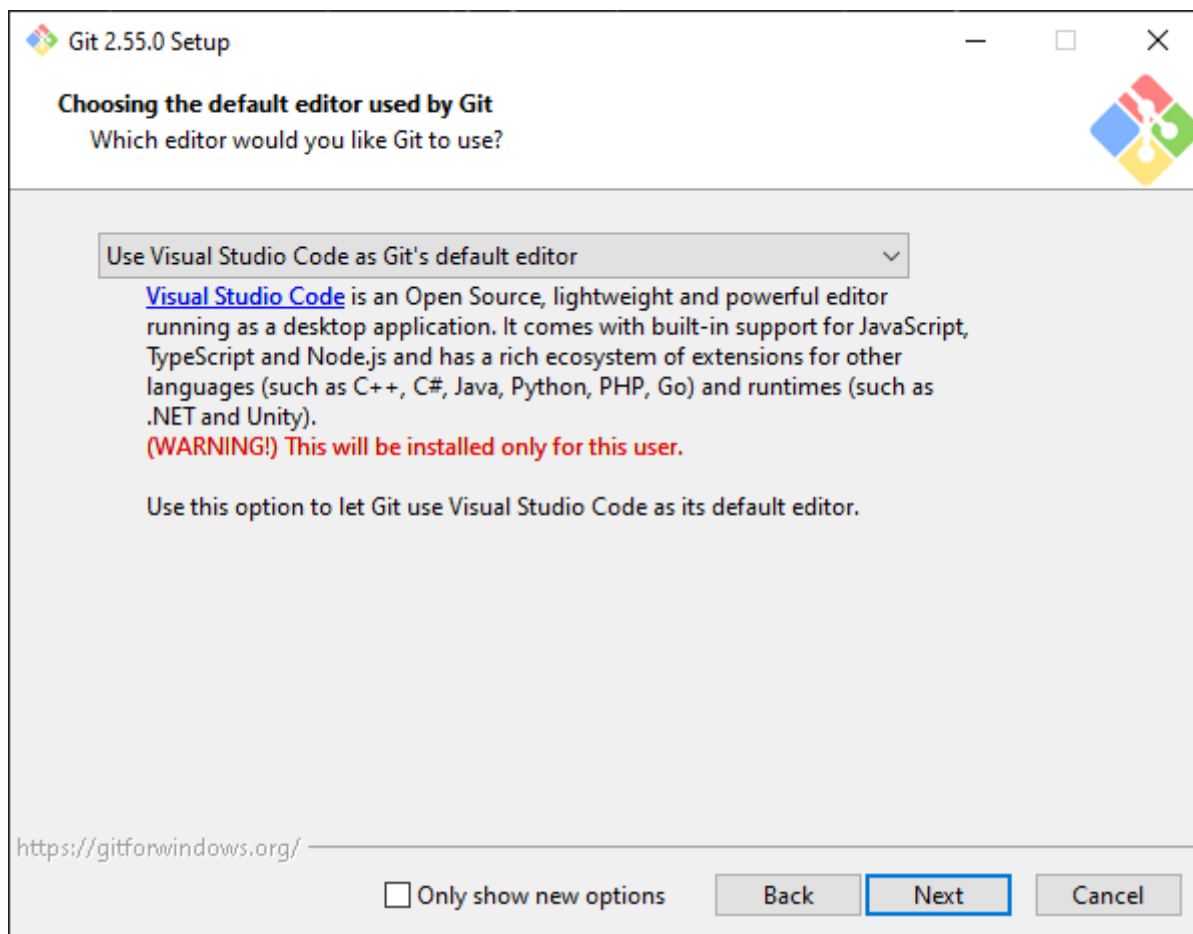


Figura 3. Instalador de Git.

Para comprobar que lo tenemos instalado correctamente, utilizaremos el comando:

```
git --version
```

Si se ha instalado correctamente, debería de salir algo así:

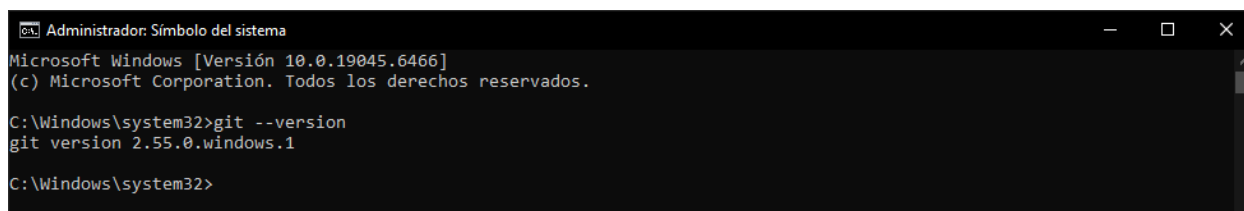


Figura 4. Resultado de la consulta de versión.

3. Montar repositorio y estructura

3.1. Configuración de variables globales

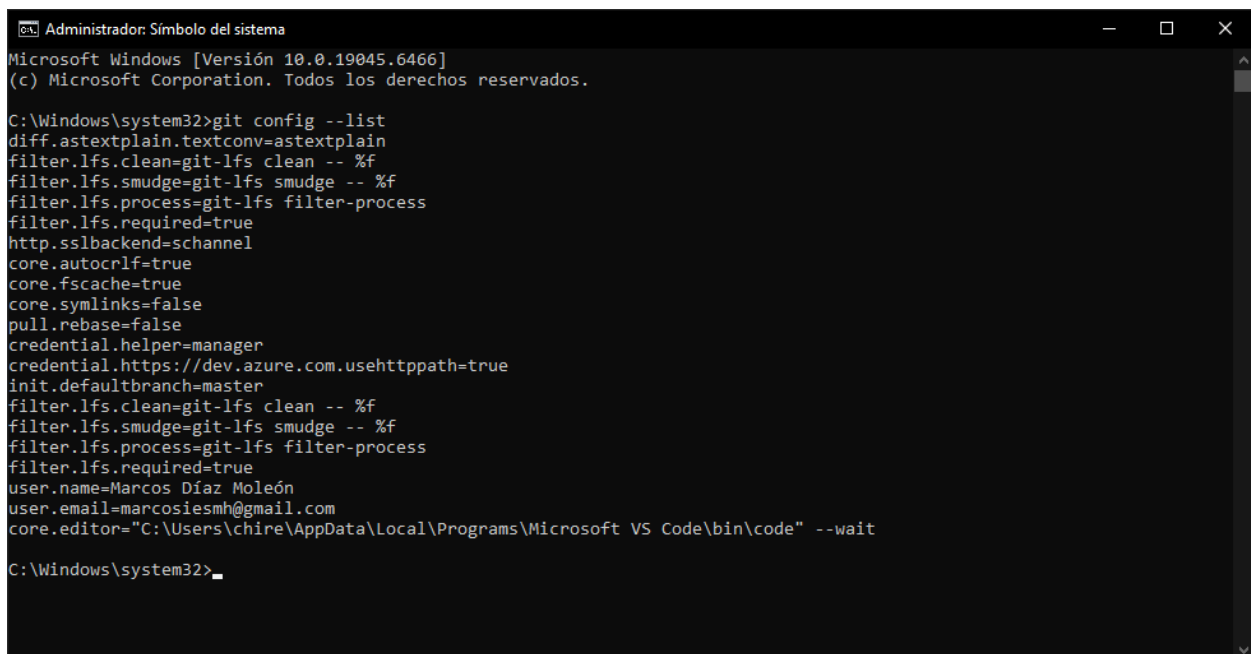
Antes de comenzar a utilizar git, deberemos de configurar las dos variables globales que son imprescindibles para poder realizar los commits, ya que no vienen configuradas por defecto. Estas se corresponden a nuestro nombre completo y email. Primero, abrimos una terminal, y a continuación ponemos los comandos:

```
git config --global user.name "Tu Nombre y Apellidos"  
git config --global user.email "Tu correo electrónico"
```

Para verificar que han sido asignadas correctamente, utilizaremos el comando:

```
git config --list
```

Deberíamos de ver nuestro nombre y correo electrónico asignados a las variables de la siguiente lista:



```
Administrador: Símbolo del sistema  
Microsoft Windows [Versión 10.0.19045.6466]  
(c) Microsoft Corporation. Todos los derechos reservados.  
  
C:\Windows\system32>git config --list  
diff.astextplain.textconv=astextplain  
filter.lfs.clean=git-lfs clean -- %f  
filter.lfs.smudge=git-lfs smudge -- %f  
filter.lfs.process=git-lfs filter-process  
filter.lfs.required=true  
http.sslbackend=schannel  
core.autocrlf=true  
core.fscache=true  
core.symlinks=false  
pull.rebase=false  
credential.helper=manager  
credential.https://dev.azure.com.usehttppath=true  
init.defaultbranch=master  
filter.lfs.clean=git-lfs clean -- %f  
filter.lfs.smudge=git-lfs smudge -- %f  
filter.lfs.process=git-lfs filter-process  
filter.lfs.required=true  
user.name=Marcos Díaz Moleón  
user.email=marcosiesmh@gmail.com  
core.editor="C:\Users\chire\AppData\Local\Programs\Microsoft VS Code\bin\code" --wait  
  
C:\Windows\system32>
```

Figura 5. Resultado de la consulta de parámetros.

3.2. Cómo trabajar con Git: Gitflow Workflow

Cuando tenemos un repositorio, la forma ideal de trabajar en él es que exista una rama principal (llamada “master”) en la que solo se debe albergar el código correcto y terminado. A partir de esta rama master, sacaremos una rama llamada “develop”, en la que iremos desarrollando características, es la común para todos los miembros del grupo de trabajo. Cuando se vaya a desarrollar un nuevo feature, lo que se debe hacer es sacar una rama de develop y en esta se implementará la nueva funcionalidad, que después se hará merge con la rama develop.

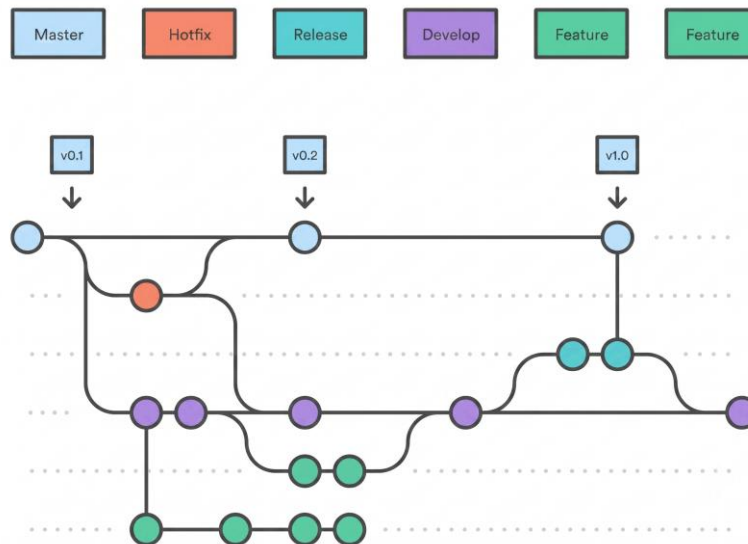


Figura 6. Workflow basado en ramas para repositorio.

3.3. Crear un repositorio

En la página principal de github, teniendo iniciada la sesión con nuestra cuenta, veremos un botón en el que pone “New”, que deberemos pulsar para comenzar la creación del repositorio:

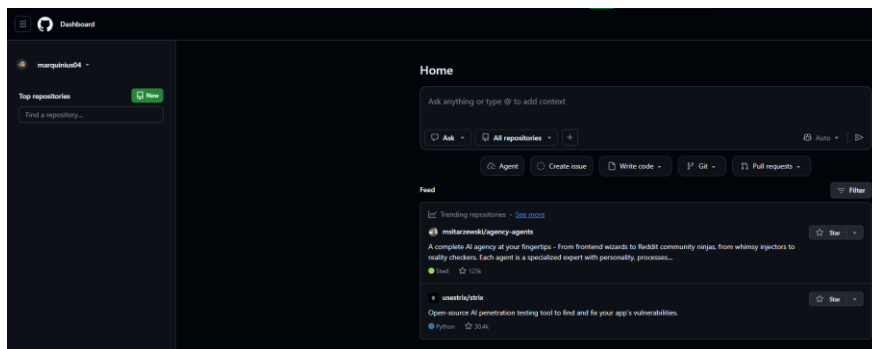


Figura 7. Crear nuevo repositorio.

En la página que se nos ha abierto ponemos los datos del repositorio y activamos el readme (archivo para la documentación), junto al .gitignore (archivo para ignorar ciertos archivos en los commits), en el que elegiremos la opción “Node”.

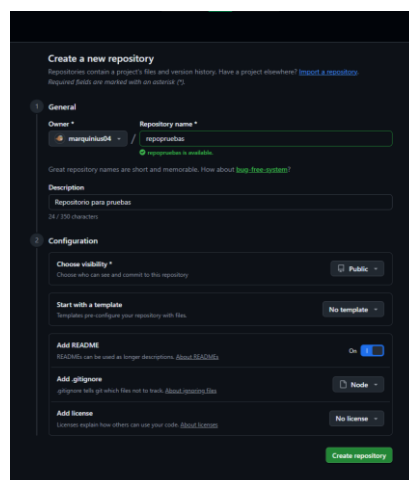
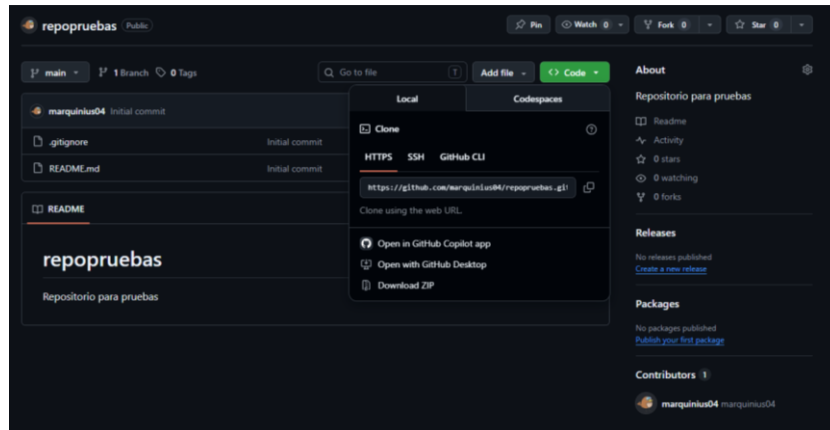


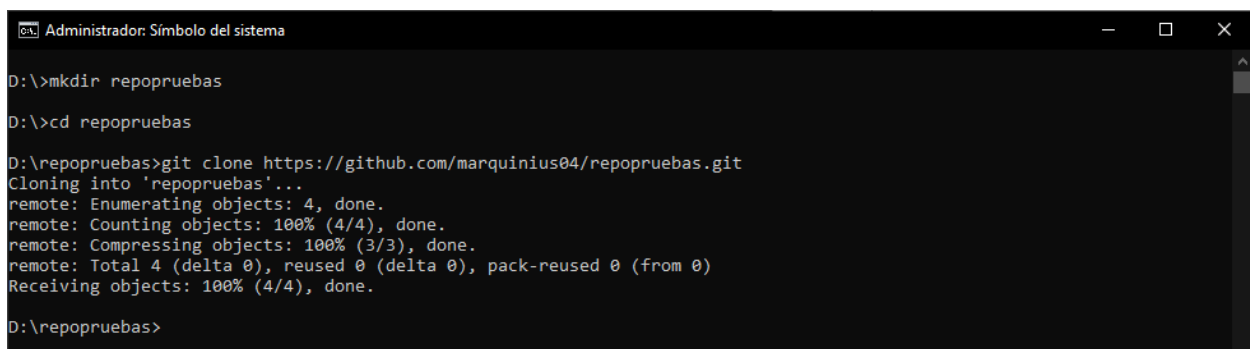
Figura 7. Opciones en la creación de un nuevo repositorio.

Tras pulsar el botón de “Create repository” ya habremos creado el repositorio y podemos clonarlo ya a nuestro ordenador. Esto lo haremos mediante el botón “Code” que aparece en la página que se nos ha redirigido:

**Figura 8.** Botón “Code” desde donde podemos clonar nuestro repositorio.

Copiaremos el enlace que pone en la pestaña de HTTPS, ya que lo vamos a utilizar para clonar el repositorio. Abrimos una terminal y creamos la carpeta donde queremos clonarlo. Dentro de la carpeta creada utilizamos el comando:

```
git clone
```

**Figura 9.** Resultado tras clonar el repositorio.

4. Trabajo en grupo

Al renombrar la carpeta del git clone, podemos clonar de nuevo el repositorio y, de esta forma es como si estuvieran trabajando dos personas:

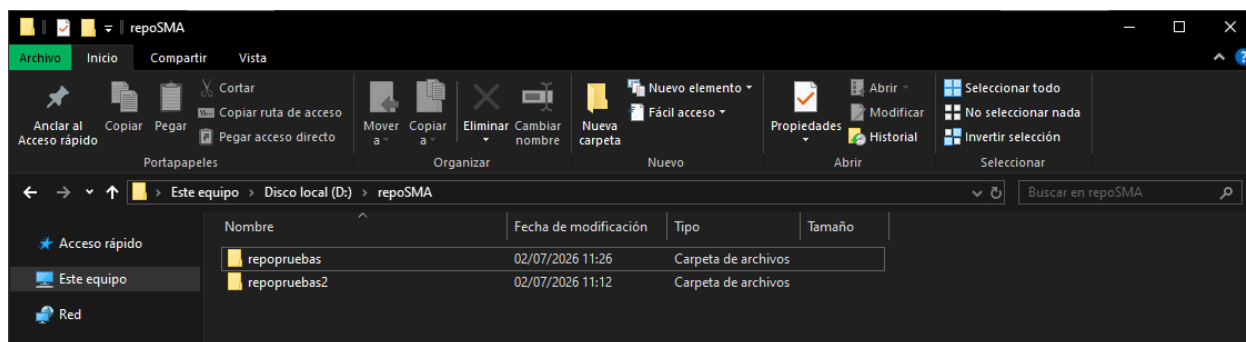


Figura 10. Duplicar carpetas para simular 2 usuarios sobre el mismo repositorio.

Abrimos cada carpeta en una ventana de Visual Studio Code (o el editor de preferencia) y creamos la estructura de nuestro proyecto con uno de los usuarios, añadiendo archivos de texto vacíos dentro de las carpetas para que estas se suban en el commit que realizaremos después.

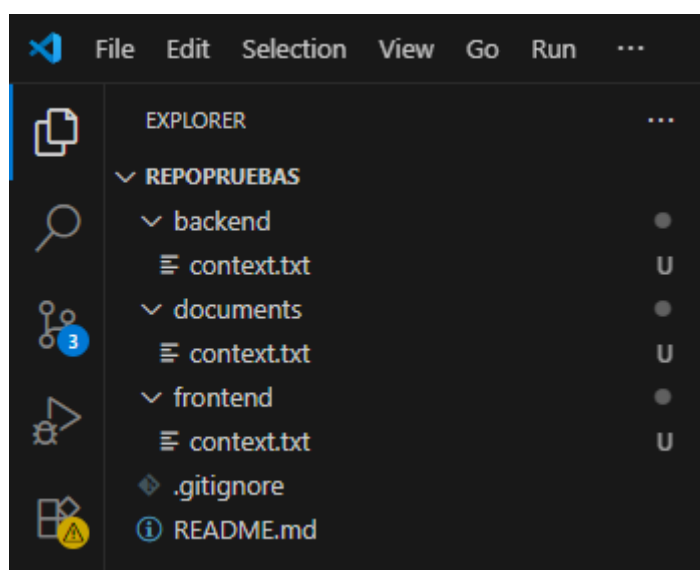
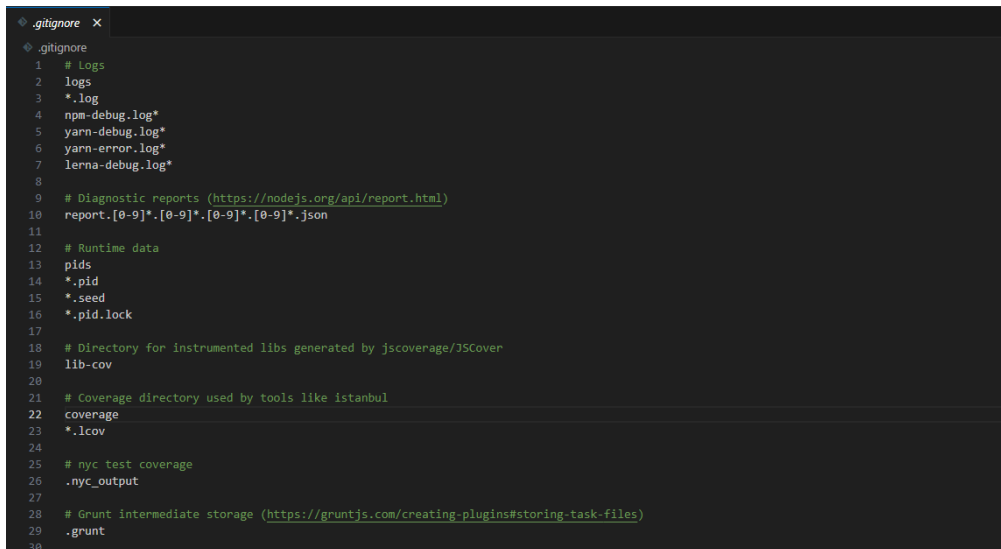


Figura 11. Ejemplo de proyecto.

Los archivos que están marcados con una “U” nos indican que están “untracked”, es decir, que no están siendo monitorizados y que no se subirán a GitHub hasta que no los indexemos.

El archivo .gitignore, como ya hemos explicado antes, contiene los nombres de los archivos del proyecto que no queremos incluir en los commits, ya sea por seguridad (api keys, contraseñas, etc...) o que no queremos que los commits sean muy pesados en tamaño.



```
.gitignore
# Logs
logs
*.log
npm-debug.log*
yarn-debug.log*
yarn-error.log*
lerna-debug.log*

# Diagnostic reports (https://nodejs.org/api/report.html)
report.[0-9]*.[0-9]*.[0-9]*.[0-9]*.json

# Runtime data
pids
*.pid
*.seed
*.pid.lock

# Directory for instrumented libs generated by jscoverage/JSCover
lib-cov

# Coverage directory used by tools like Istanbul
coverage
*.lcov

# nyc test coverage
.nyc_output

# Grunt intermediate storage (https://gruntjs.com/creating-plugins#storing-task-files)
.grunt
```

Figura 12. Contenido de `.gitignore`.

Para preparar los archivos antes de guardarlos, se deben indexar. Esto se puede realizar en consola mediante el comando:

```
git add .
```

O utilizando la pestaña de “Source control” en la barra lateral de VS Code, pulsando el botón de “Stage all changes” para que los archivos pasen a estar monitorizados:

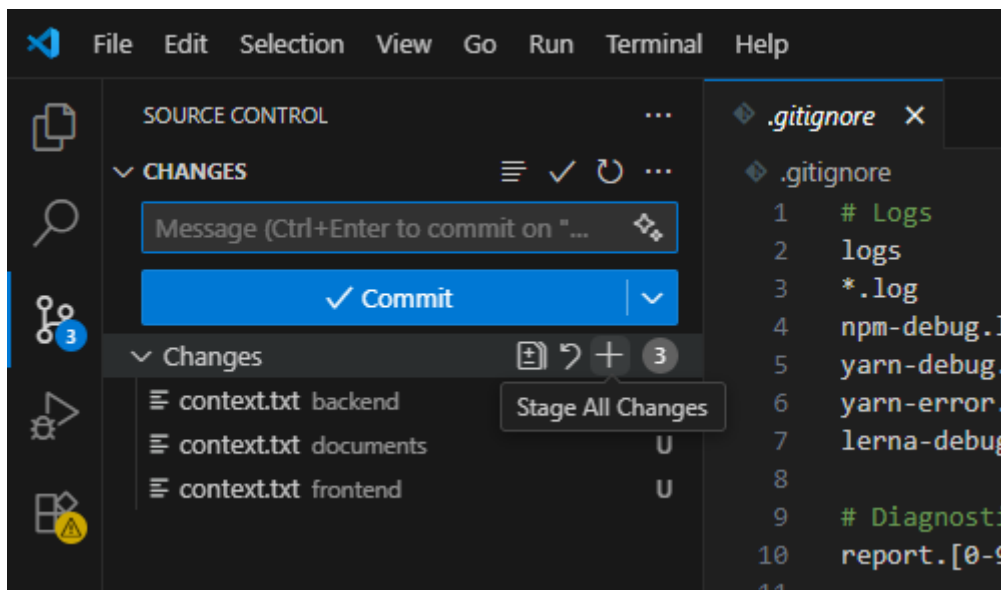


Figura 13. Indexar los archivos para su utilización en git.

Para crear el commit podemos realizarlo mediante la terminal con el comando:

```
git commit -m "Primer commit"
```

O directamente poniendo el nombre del commit en la caja de texto superior de “Source control” y presionando **Control+Enter**.

Para comprobar que el commit ha sido realizado correctamente, en la consola de comandos empleamos el comando:

```
git log
```

Debería salir algo similar a esto:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

indows [Versión 10.0.19045.6466]
(c) Microsoft Corporation. Todos los derechos reservados.

D:\repoSMA\repopruebas>git log
commit 07731efb67c07a05af285f4c986bafcf614d414c (HEAD -> main)
Author: Marcos Díaz Moleón <marcosiesmh@gmail.com>
Date: Thu Jul 2 11:57:45 2026 +0200

    Primer commit

commit 3e1ecf8cd6911f477161b652495e3f1c1be654eb (origin/main, origin/HEAD)
Author: marquinius04 <marcosiesmh@gmail.com>
Date: Thu Jul 2 11:04:14 2026 +0200

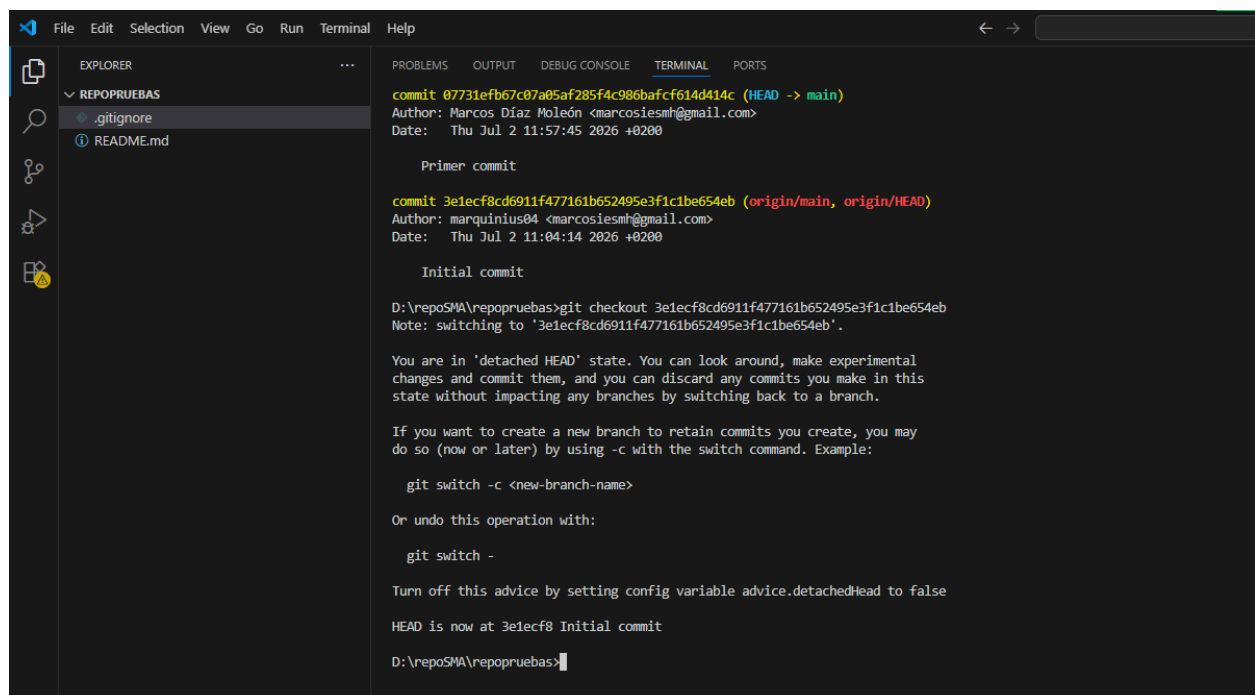
    Initial commit

D:\repoSMA\repopruebas>
```

Figura 14. Resultado de la consulta de git log.

Sabemos que nuestro commit se ha realizado correctamente ya que observamos el nombre que le habíamos puesto en la lista del “git log”.

También podemos volver a versiones pasadas utilizando el comando git checkout con el identificador del commit al que queremos regresar. En este caso vemos que hay uno que es el commit de la creación del repositorio, usaremos ese para comprobar cómo funciona este comando.



```
File  Edit  Selection  View  Go  Run  Terminal  Help

EXPLORER
REPOPRUEBAS
  .gitignore
  README.md

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

commit 07731efb67c07a05af285f4c986bafcf614d414c (HEAD -> main)
Author: Marcos Díaz Moleón <marcosiesmh@gmail.com>
Date: Thu Jul 2 11:57:45 2026 +0200

    Primer commit

commit 3e1ecf8cd6911f477161b652495e3f1c1be654eb (origin/main, origin/HEAD)
Author: marquinius04 <marcosiesmh@gmail.com>
Date: Thu Jul 2 11:04:14 2026 +0200

    Initial commit

D:\repoSMA\repopruebas>git checkout 3e1ecf8cd6911f477161b652495e3f1c1be654eb
Note: switching to '3e1ecf8cd6911f477161b652495e3f1c1be654eb'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

    git switch -c <new-branch-name>

Or undo this operation with:

    git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at 3e1ecf8 Initial commit

D:\repoSMA\repopruebas>
```

Figura 15. Resultado de cambiar el repositorio a otro commit.

Observamos que, tras utilizar el git checkout con el id del primer commit al repositorio, efectivamente desaparecen las carpetas que habíamos creado. Para regresar al commit más reciente usamos uno de estos

comandos (depende de como se llame tu rama principal):

```
git checkout master  
git checkout main
```

Ahora que estamos de vuelta en la rama principal y en el último commit, queremos realizar la subida del commit para que todos nuestros compañeros puedan acceder a los nuevos cambios. Antes de esto deberemos comprobar que estamos en la rama que queremos, mediante el comando:

```
git branch
```

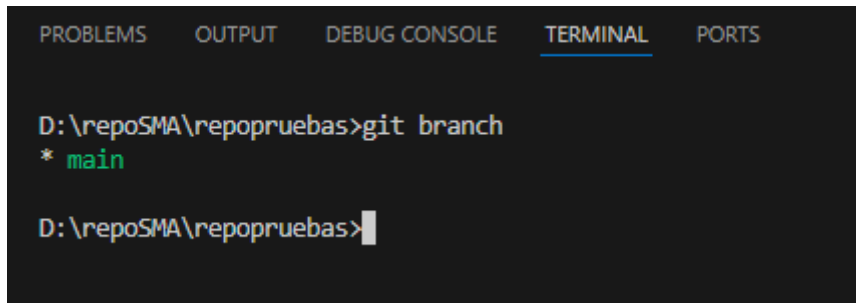


Figura 16. Resultado de ejecutar `git branch`.

La rama activa es la que se resalta en verde y con un asterisco. Ahora que ya hemos comprobado que estamos en la rama correcta, usaremos el comando:

```
git push
```

Para subir el commit al repositorio en la nube en GitHub.

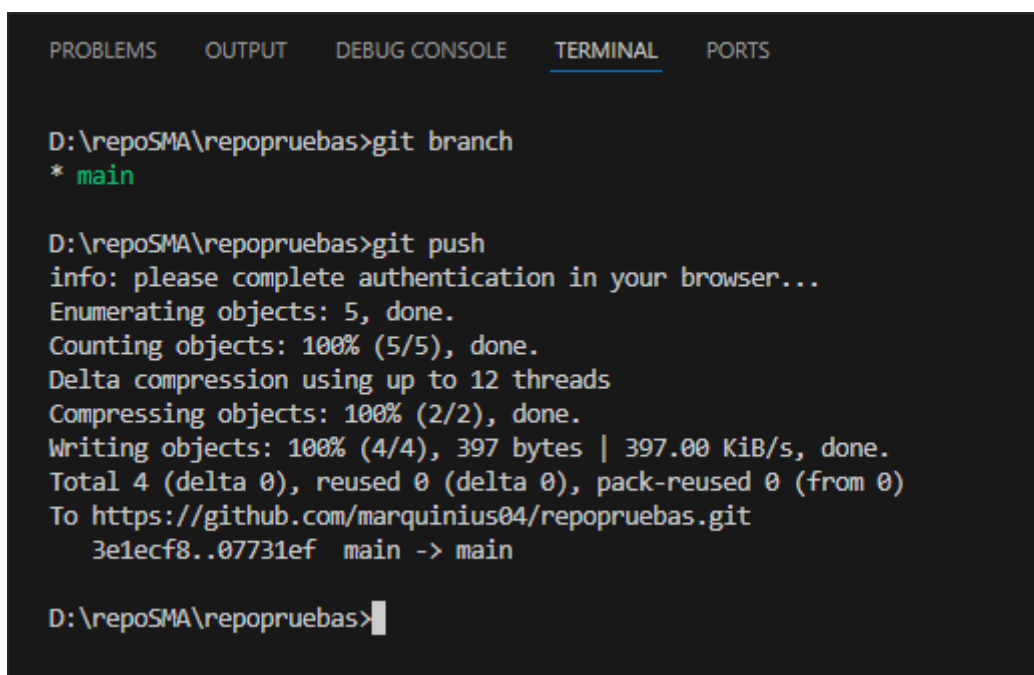


Figura 17. Resultado de subir el código local al repositorio remoto.

Ahora iremos a la página de nuestro repositorio para comprobar que los cambios se han subido.

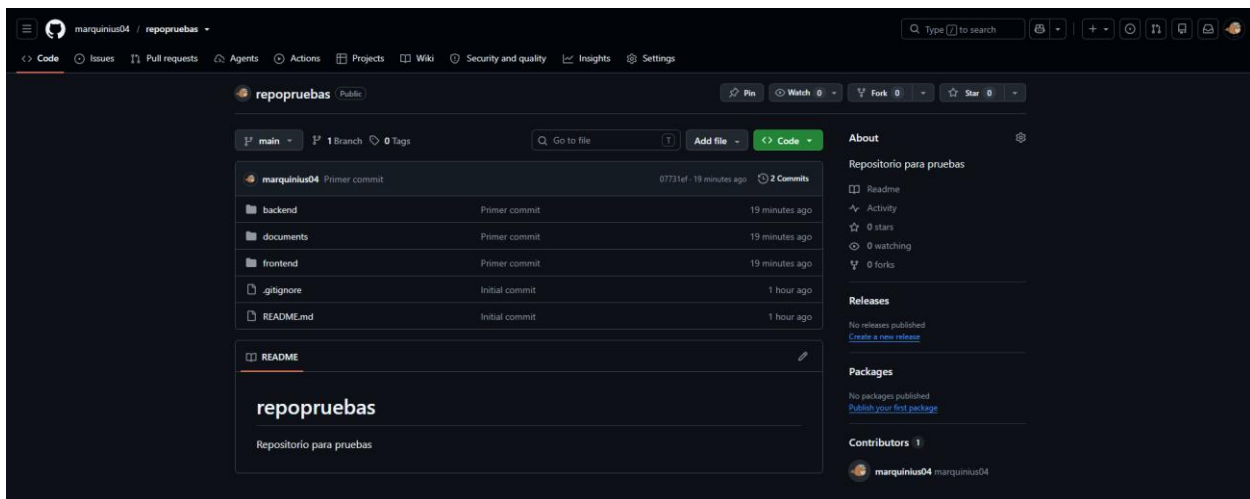


Figura 18. Estado del repositorio remoto tras la subida de contenidos locales.

Si los cambios no aparecen, hay que refrescar la página.

4.1. Sincronizar ramas

El usuario 2 todavía no tiene los cambios que ha realizado el usuario 1, por lo que tiene que utilizar el comando “git branch” para comprobar que está en la rama correcta y, si lo está, utilizar el comando:

```
git pull
```

Que sirve para extraer la última versión con todos los cambios de la rama en la que te encuentras.

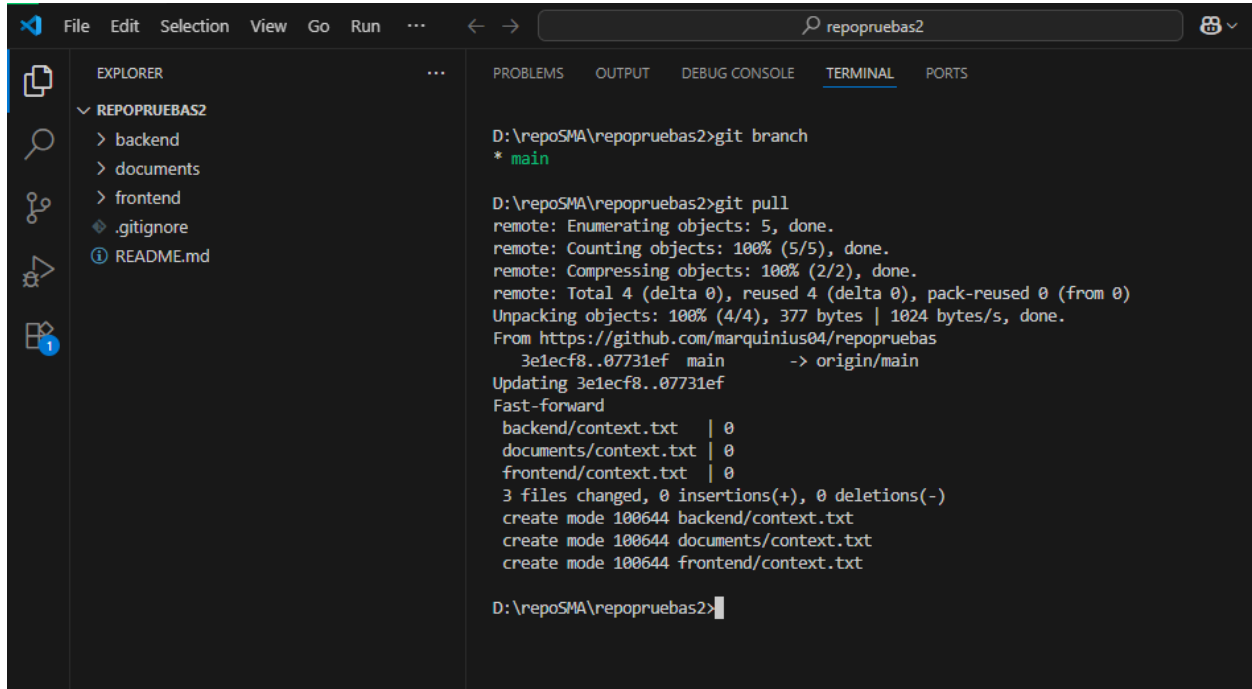


Figura 19. Resultado de ejecutar git pull.

4.2. Creación rama “develop”

Como hemos explicado antes en la sección de GitFlow Workflow, debemos crear una rama llamada develop

para integrar las características antes de enviarlas a producción.

El usuario 1 crea la rama ejecutando el comando:

```
git branch develop
```

Y luego cambia a ella con git checkout.

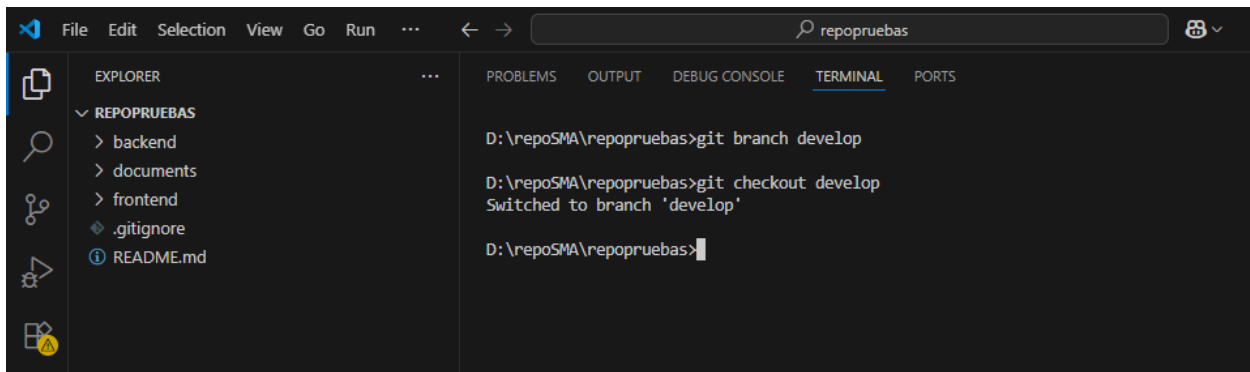


Figura 20. Crear y cambiar a la nueva rama.

El usuario 1 crea en la carpeta documents un archivo llamado tareas.txt, donde define el reparto de tareas.

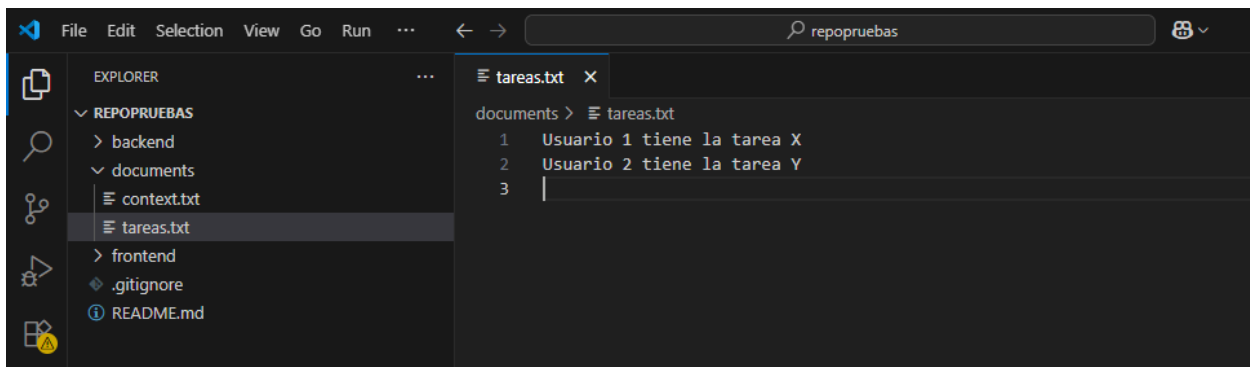


Figura 21. Crear archivo de tareas.txt con contenido.

El usuario 1 indexa el archivo, ya que está “untracked”, mediante “git add .” o la interfaz de VS Code. Realiza el commit y ejecuta “git push”. Al ser una rama recién creada, git nos pedirá que utilicemos el comando siguiente para que enlacemos la rama local con la remota:

```
git push --set-upstream origin develop
```

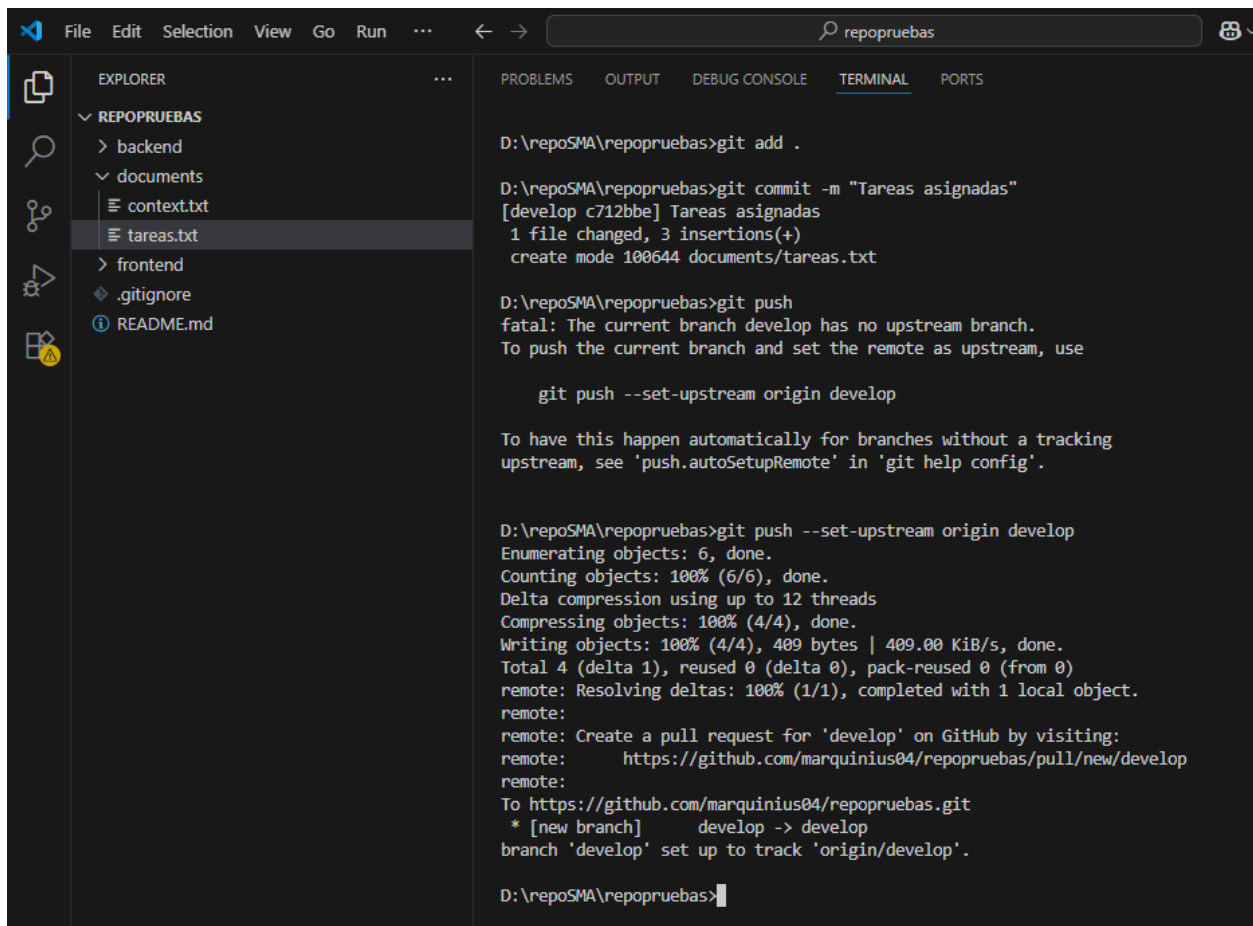


Figura 22. Indexar el archivo tareas.txt y enlace a la rama remota.

4.3. Sincronización de la rama develop

Para que el usuario 2 pueda ver la nueva rama develop creada en el servidor, debe actualizar su información local ejecutando el comando:

```
git fetch
```

Al hacer:

```
git branch -a
```

ya podrá ver la rama remota. Finalmente, ejecuta *git checkout develop* para crear su rama local enlazada automáticamente a la remota y obtener el archivo tareas.txt:

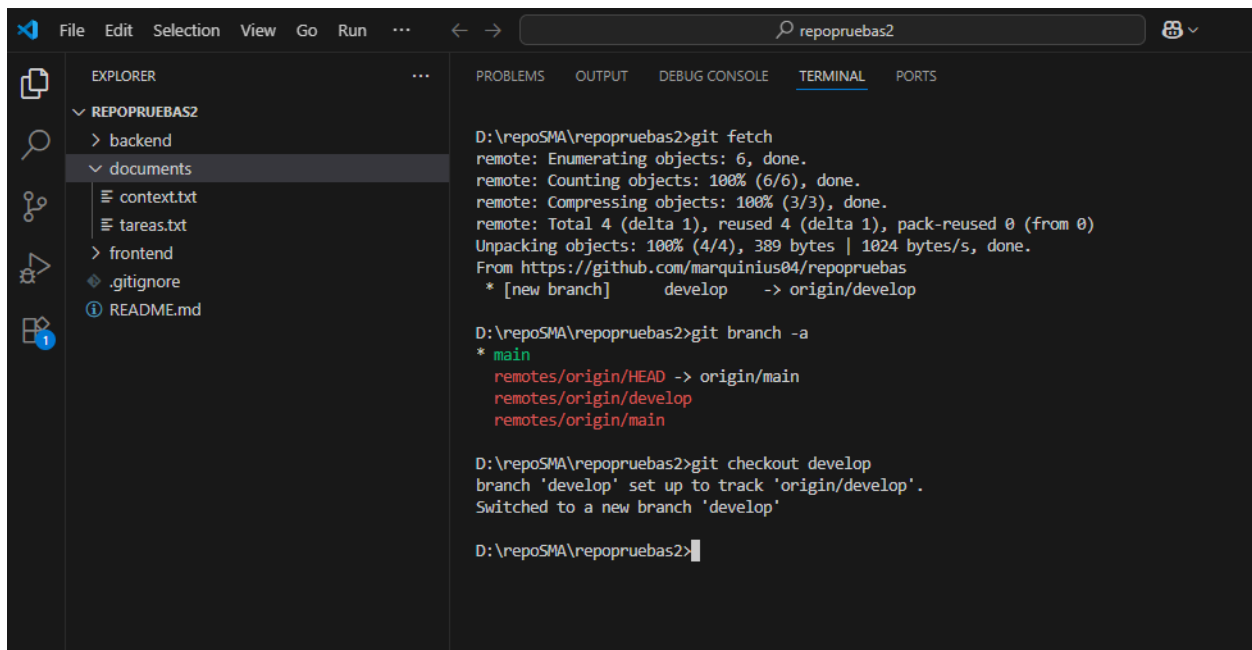


Figura 23. Cambiar a la nueva rama develop.

4.4. Trabajo paralelo en ramas de features

Cada programador debe trabajar en una rama aislada para desarrollar su funcionalidad específica sin interferir en el código de su compañero.

El usuario 1 crea y salta a su rama de trabajo para el backend mediante los comandos:

```
git branch back_feature1
git checkout back_feature1
```

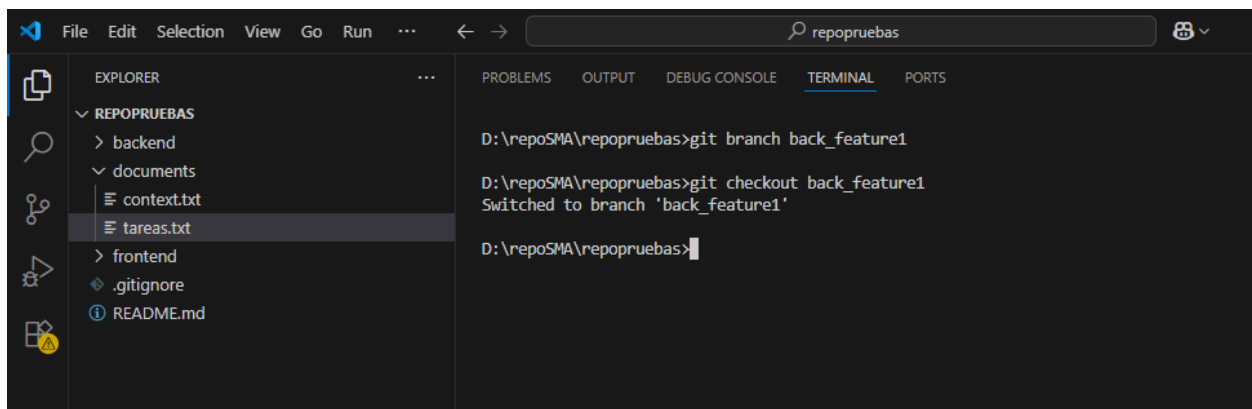


Figura 24. Resultado de crear y cambiar la rama back_feature1.

El usuario 2 utiliza el comando rápido para crear y cambiar de rama en un solo paso (equivalente a los dos comandos anteriores):

```
git checkout -b front_feature1
```

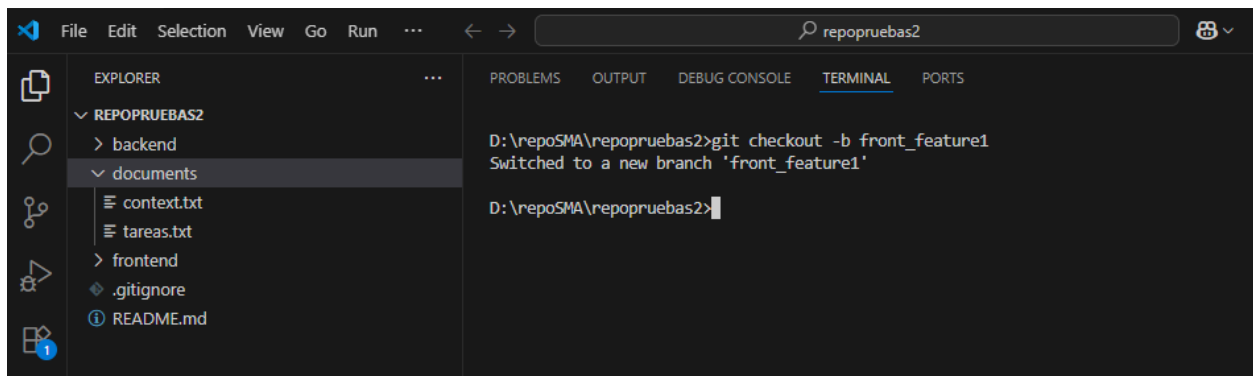


Figura 24. Resultado de crear y cambiar la rama `front_feature1`.

El usuario 1 crea el código del backend en `codigo_back.txt` y edita el archivo común `tareas.txt` para marcar su tarea 1 como “cumplida”.

El usuario 2 crea el código del frontend en `codigo_front.txt` y modifica también `tareas.txt` para marcar su tarea 2 como “cumplida”.

Ambos usuarios guardan los cambios realizados y los indexan mediante “`git add .`” o mediante la interfaz de VS Code. Cada uno realiza su commit con sus respectivos nombres.

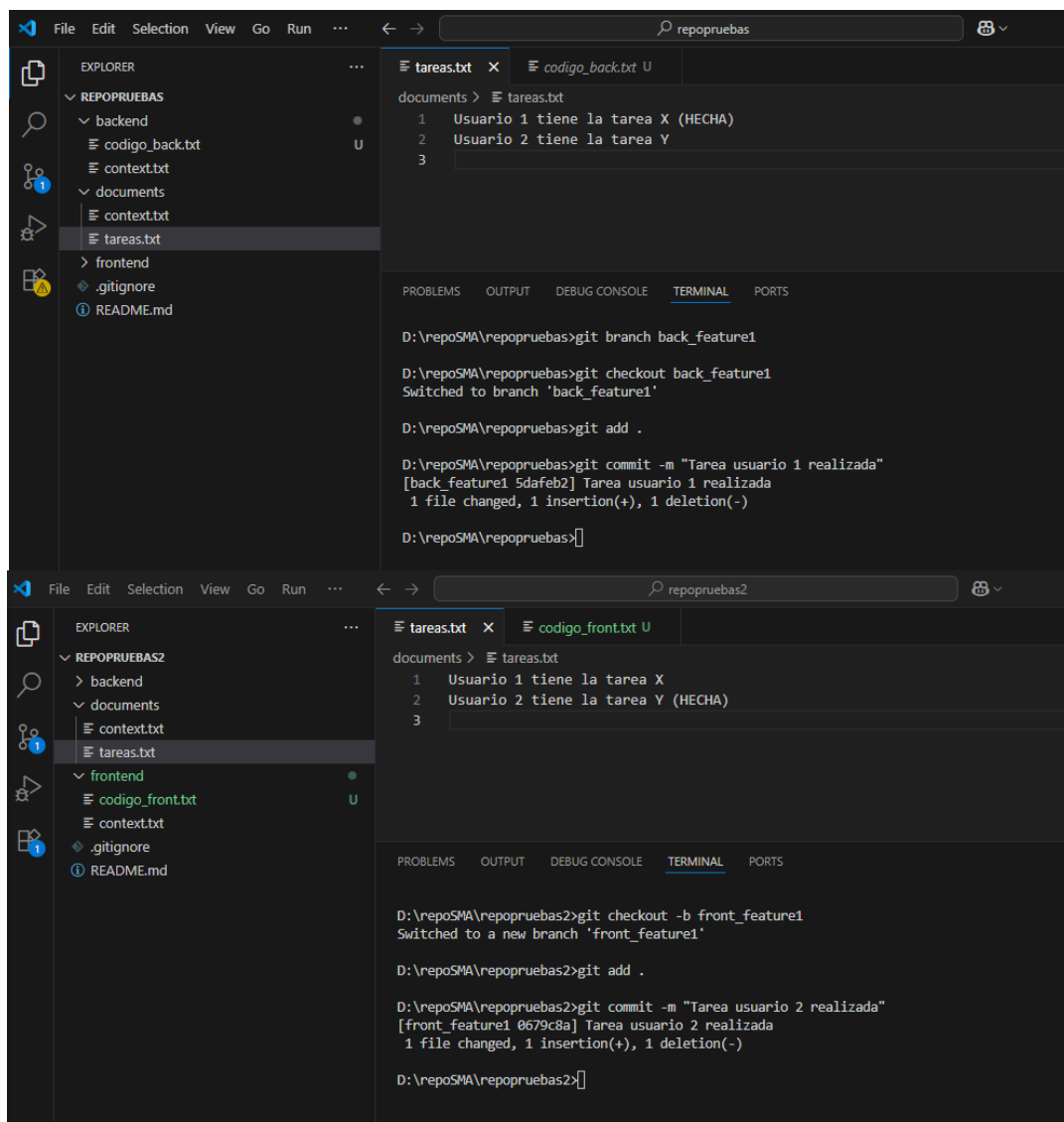


Figura 25. Resultado de sincronizar las ramas con el repositorio.

4.5. Fusión de la rama de backend

Cuando un “feature” o desarrollo está listo y estable, se debe integrar de vuelta en la rama común de desarrollo (develop). El usuario 1 se posiciona en la rama de destino mediante “git checkout develop”.

Ejecuta el comando:

```
git merge back_feature1
```

Para traer todos los archivos de back_feature1 y los cambios de su tarea a la rama local develop.

Ejecuta git push para actualizar la rama develop remota en el servidor, haciendo que el trabajo de back_feature1 sea accesible para el resto del equipo.

Después volvemos a la rama back_feature1 con git checkout y le damos al botón de “Publish branch”.

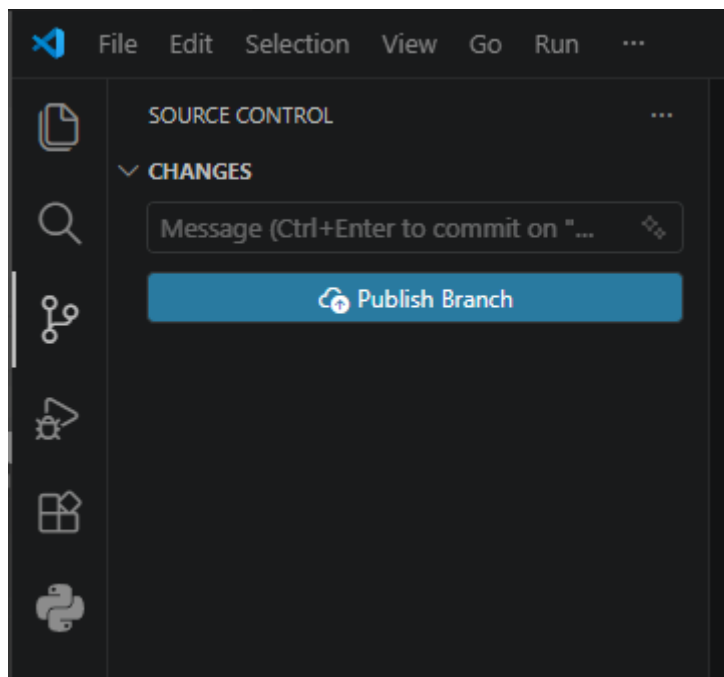


Figura 26. Botón Publish branch.

4.6. Fusión de la rama de frontend y resolución de conflictos

Esta sección aborda el escenario donde la fusión automática falla porque ambos usuarios han modificado las mismas líneas de un archivo común.

4.6.1. Descarte de errores comunes

Si por error el usuario 2 comenzó a modificar archivos directamente en la rama develop local sin estar en su rama de características, debe descartar los cambios locales indeseados desde la interfaz gráfica de VS Code o mediante comandos para volver a un estado limpio.

4.6.2. Sincronización previa obligatoria

El usuario 2 cambia a develop. Git le advertirá que su rama local está por detrás (behind) de la remota. Debe ejecutar “git pull” para descargar la fusión de backend que subió el usuario 1 antes de intentar fusionar su

propio trabajo.

4.6.3. Fusión con conflicto

El usuario 2 ejecuta:

```
git merge front_feature1
```

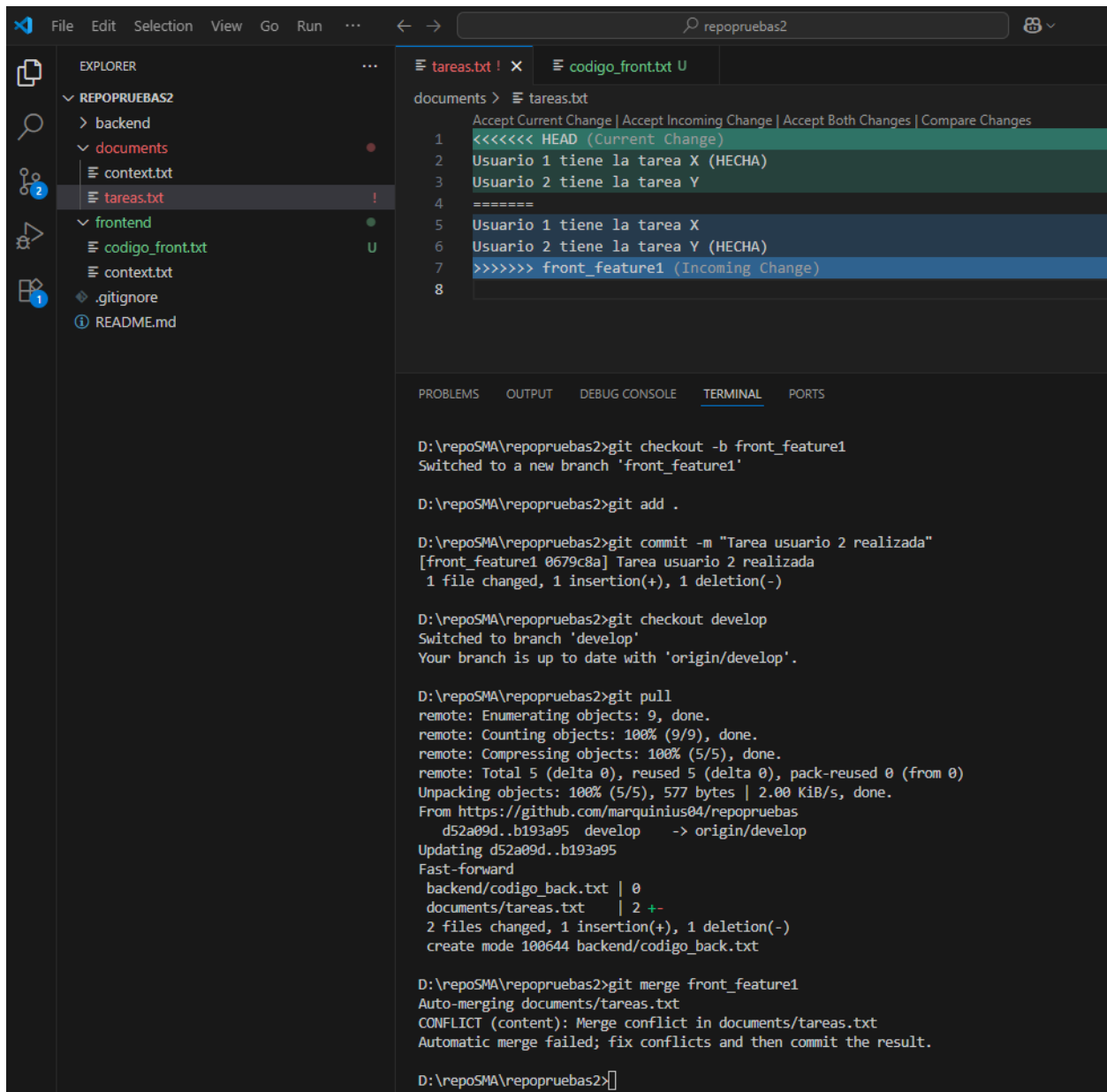


Figura 27. Acción de merge.

Al intentar unificar el archivo tareas.txt donde ambos cambiaron las líneas de estado de tareas, Git detiene el proceso y avisa que el merge automático ha fallado por un conflicto.

4.6.4. Resolución manual del conflicto en VS Code

Para resolver este conflicto, el usuario debe editar manualmente el archivo eliminando los marcadores de conflicto de Git y dejando el texto final correcto con los cambios de los dos usuarios.

4.6.5. Finalización de la fusión

Después de esto guardamos el archivo, utilizar el comando “git add .” para indexar el archivo de nuevo y hacemos un commit llamado “Resolución de conflictos merge front_feature1 en develop”. Para terminar, subimos la rama actualizada al servidor remoto de GitHub con “git push origin front_feature1”.

5. Trabajo con Visual Studio Code

Los comandos de git se pueden realizar también mediante la interfaz de Visual Studio Code, y es lo que vamos a ver a continuación.

5.1. Crear una rama

Para crear una rama utilizaríamos el comando “git branch nombre” en la terminal, pero para hacerlo desde el menú de VS Code, presionamos “CONTROL + SHIFT + P” y se nos abrirá un menú así:

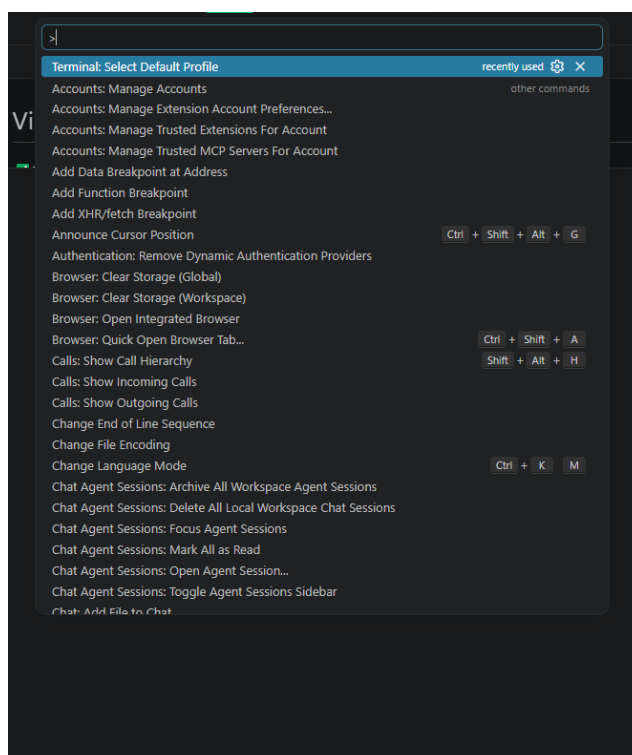


Figura 28. Menú para insertar comandos.

A continuación escribiremos el comando en el campo de texto del menú y seleccionaremos la opción de “Git: Create Branch” y le pondremos el nombre que queramos:

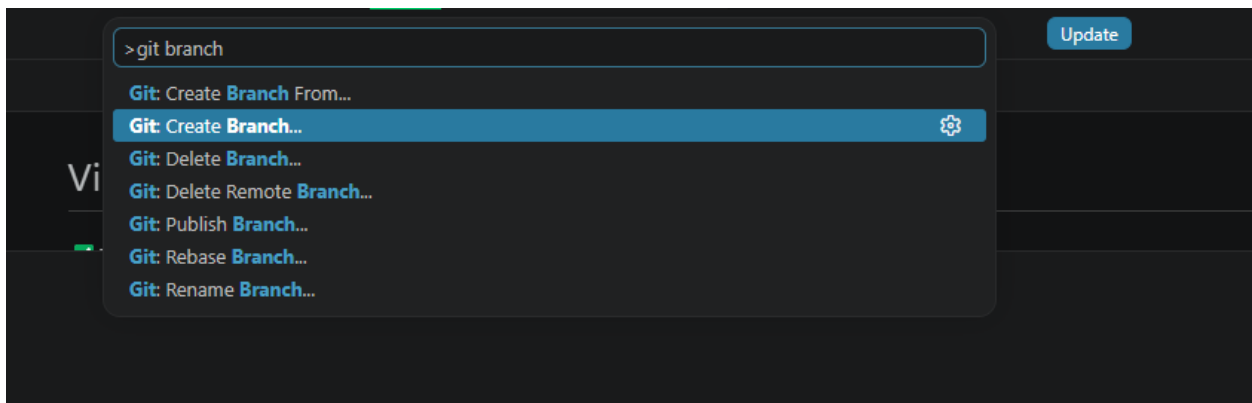


Figura 29. Crear nueva rama desde el menú de VS Studio.

5.2. Cambiar de rama

Para acceder a nuestra nueva rama, utilizaremos de nuevo el mismo menú con “CONTROL + SHIFT + P” y escribiremos git checkout en el campo de texto:

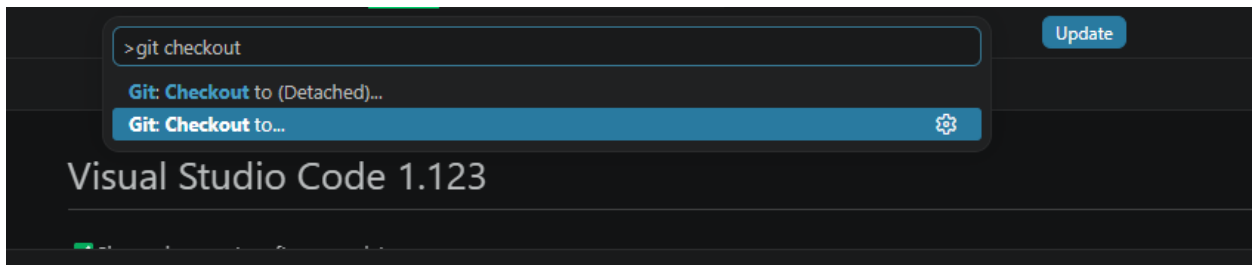


Figura 30. Cambiar de rama desde el menú de VS Studio.

Tras seleccionar la opción de “Git: Checkout to”, seleccionamos la rama a la que queremos cambiarnos.

5.3. Guardar cambios

Para realizar un commit de los cambios locales, habría que abrir el mismo menú de nuevo y seleccionar la opción “Git: Commit”, te pedirá el nombre del commit a realizar y se guardará.

5.4. Subir cambios al servidor

Para subir los commits, se abriría el mismo menú y se seleccionaría la opción de “Git: Push”, que subirá todos los commits locales a GitHub.

5.5. Descargar cambios del servidor

Se utilizaría el git pull, que en el caso del menú de VS Code sería la opción de “Git: Pull”. VS Code extraerá los últimos cambios del repositorio remoto y los fusionará en tu rama actual en un solo paso.

5.6. Actualizar el mapa del repositorio

Abre la paleta de comandos y busca Git: Fetch (o Git: Recuperar). Esta opción le pregunta a GitHub si hay

cambios o nuevas ramas de tus compañeros sin tocar tus archivos de código locales, lo cual te permite revisar el historial antes de hacer un pull.

6. Plugin Gitgraph

Para facilitar el seguimiento visual del proyecto y asegurar que todas las integraciones se han realizado de manera limpia, existen herramientas visuales tanto locales como en el servidor remoto que permiten inspeccionar el árbol de confirmaciones (commits). Esta extensión de VS Code se puede instalar en el apartado de extensiones, buscando el nombre “git graph”.

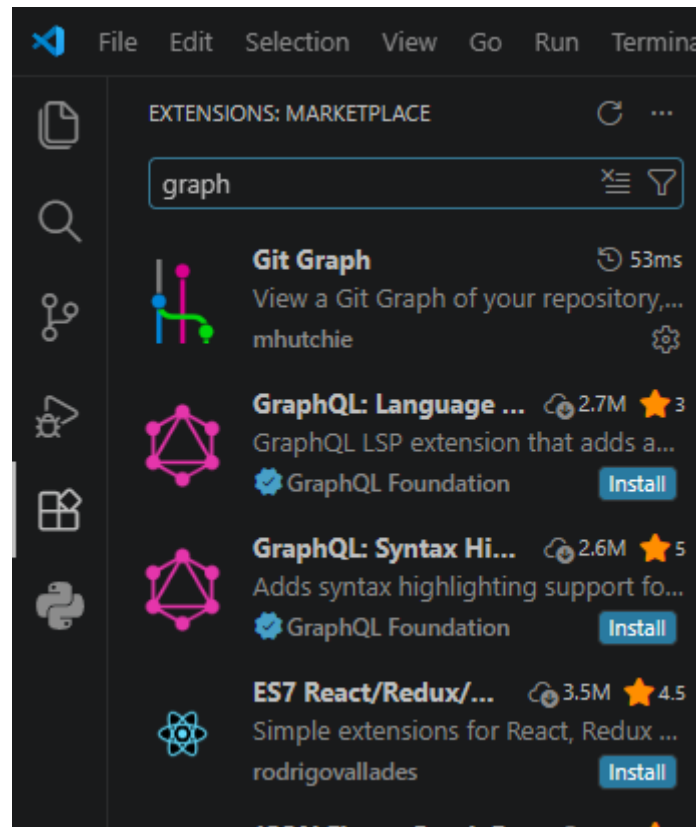


Figura 31. Plugin Git Graph.

Una vez instalado el plugin, aparecerá un pequeño icono de git graph en la barra de abajo de VS Code:

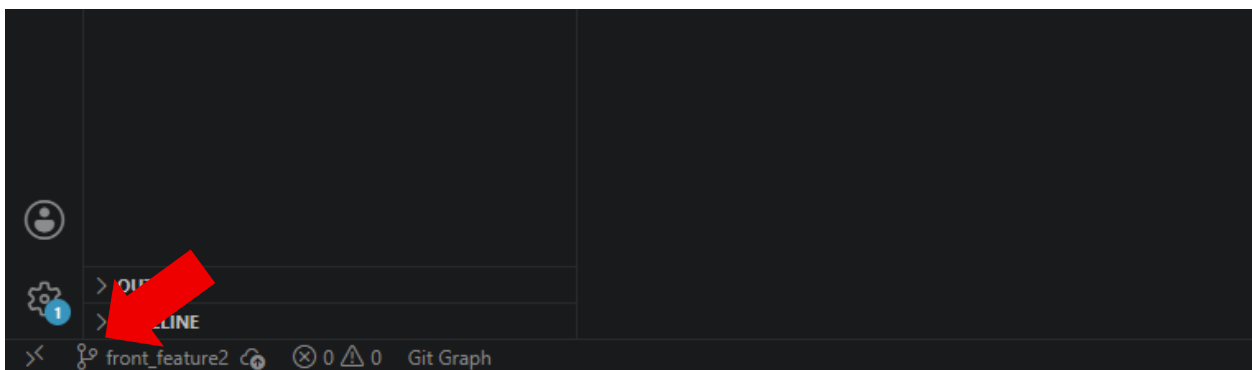


Figura 31. Icono de Git Graph.

Si pulsamos donde pone “Git Graph”, se abrirá el siguiente menú con el gráfico que representa con líneas de diferentes colores la bifurcación de la rama principal en las ramas que hemos creado.



Figura 32. Menú de Git Graph.

GitHub también ofrece su propia herramienta web para analizar cómo se relacionan los envíos de los diferentes colaboradores del equipo. Dentro de la página del repositorio en GitHub, se debe navegar a la pestaña Insights (Información) situada en la barra superior del proyecto.

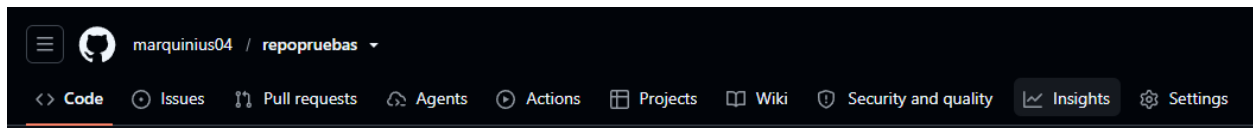


Figura 33. Barra de menú de GitHub con la opción Insights la segunda por la derecha.

A continuación, seleccionar la opción Network (Red) en el menú de la izquierda.

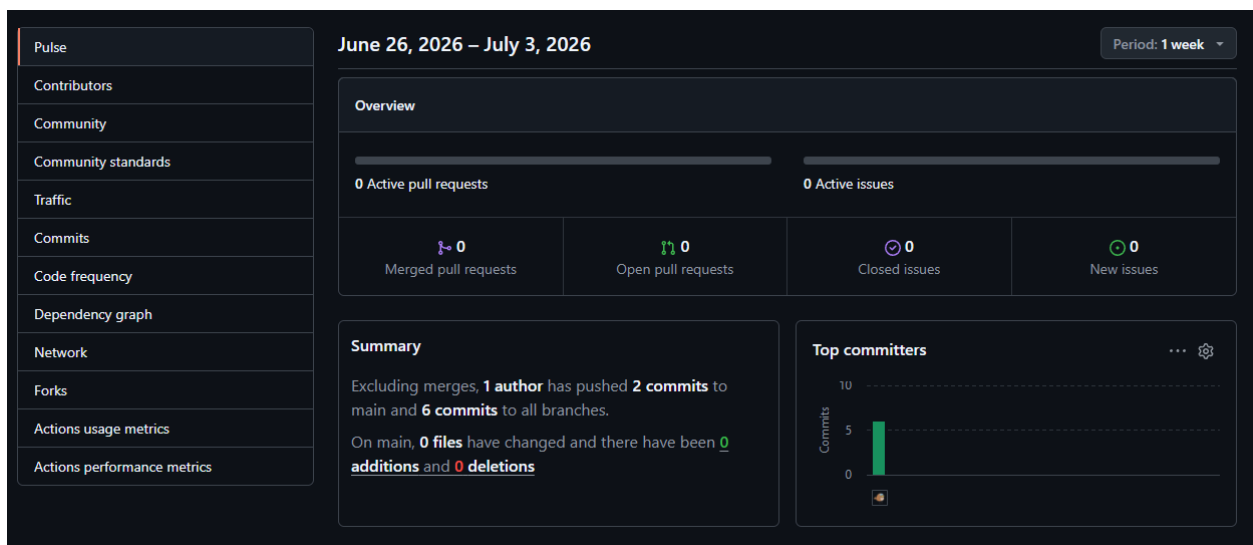


Figura 34. Opción Network de GitHub.

Tras seleccionar el menú Network podremos ver el mapa:

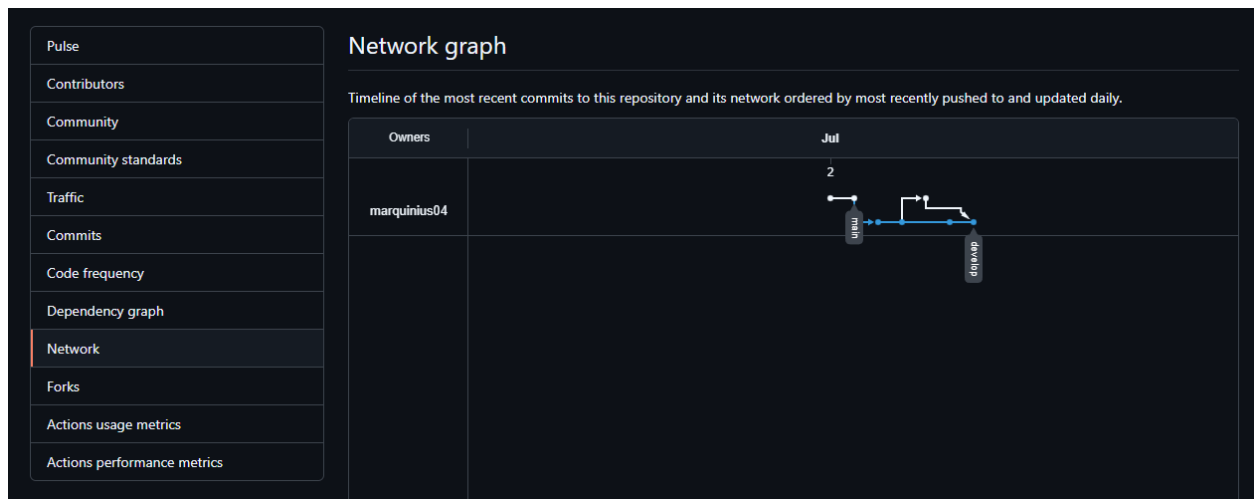


Figura 35. Mapa de acciones sobre las ramas.

7. Ver y borrar ramas

La eliminación de una rama que ya ha sido integrada consta de dos fases obligatorias: borrar la rama en el servidor de GitHub (remoto) y borrar la rama en tu computadora (local), ya que realizar una acción no borra automáticamente la otra.

7.1. Sincronización y comprobación inicial

Antes de borrar nada, asegúrate de estar situado en la rama develop, tener todos tus cambios al día y haber descargado la última versión que incluya las fusiones de todo el equipo:

1. Cambia a develop.
2. Asegura la sincronización con un pull.

```
D:\repoSMA\repopruebas>git checkout develop
Already on 'develop'
Your branch is up to date with 'origin/develop'.

D:\repoSMA\repopruebas>git pull
Already up to date.
```

Figura 36. Comprobación inicial.

7.2. Borrar la rama local

Para borrar la rama a nivel local, utilizaremos el comando “git branch -d nombredelarama”, el “-d” es el que indica que queremos borrar la rama del comando.

```
D:\repoSMA\repopruebas>git branch -d back_feature1
warning: deleting branch 'back_feature1' that has been merged to
'refs/remotes/origin/back_feature1', but not yet merged to HEAD
Deleted branch back_feature1 (was 5dafeb2).

D:\repoSMA\repopruebas>git branch
* develop
main
```

Figura 37. Borrado local de una rama.

7.3. Borrar la rama en el servidor remoto (GitHub)

Para borrar una rama directamente en el servidor de GitHub desde tu terminal local, se utiliza una sintaxis especial del comando git push empleando el símbolo de dos puntos ":" antes del nombre de la rama remota. Esto le indica a Git que envíe una instrucción de eliminación al servidor.

```
D:\repoSMA\repopruebas>git push origin :back_feature1
To https://github.com/marquinius04/repopruebas.git
- [deleted] back_feature1
```

Figura 37. Borrado de la rama remota.

Comprobamos que la rama realmente ha sido borrada del repositorio en la página del mismo.

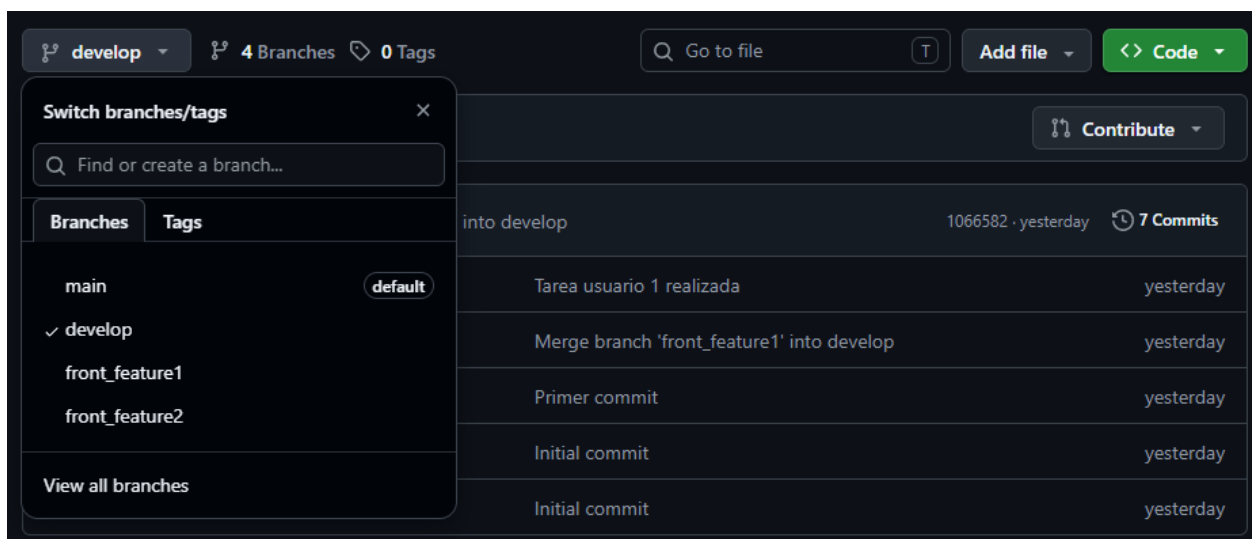


Figura 38. Comprobación del repositorio remoto.

Como podemos observar, ya no está la rama de back_feature1. Repetiremos lo mismo con las ramas del usuario 2.

7.4. Limpieza de referencias locales

Si al hacer "git branch -a" siguen apareciendo las ramas que borramos en el apartado anterior, solamente habría que utilizar git fetch con el parámetro "-p", de forma que quedaría.

```
git fetch -p
```

```
D:\repoSMA\repopruebas>git branch -a
* develop
main
remotes/origin/HEAD -> origin/main
remotes/origin/develop
remotes/origin/front_feature1
remotes/origin/front_feature2
remotes/origin/main

D:\repoSMA\repopruebas>git fetch -p
From https://github.com/marquinius04/repopruebas
- [deleted]          (none)      -> origin/front_feature1
- [deleted]          (none)      -> origin/front_feature2

D:\repoSMA\repopruebas>git branch -a
* develop
main
remotes/origin/HEAD -> origin/main
remotes/origin/develop
remotes/origin/main
```

Figura 39. Resultado de actualizar ramas.

7.5. Paso a producción

Cuando todas las características han sido probadas y fusionadas de forma estable dentro de la rama develop, llega el momento de volcar todo ese trabajo en la rama principal o de producción (master). En este punto, master se encuentra vacía o desactualizada, mientras que develop contiene todos los avances unificados (tareas.txt, códigos de backend y frontend).

Para realizar este despliegue de forma segura, ejecuta los siguientes comandos en orden:

1. Cambiar a la rama de producción (main o master):

```
git checkout main
```

2. Asegurar la actualización de la rama destino:

```
git pull
```

3. Realizar la fusión de desarrollo en master:

```
git merge develop
```

4. Publicar la rama actualizada en GitHub:

```
git push
```

```
D:\repoSMA\repopruebas>git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

D:\repoSMA\repopruebas>git pull
Already up to date.

D:\repoSMA\repopruebas>git merge develop
Updating 07731ef..1066582
Fast-forward
 backend/codigo_back.txt | 0
 documents/tareas.txt    | 2 ++
 2 files changed, 2 insertions(+)
 create mode 100644 backend/codigo_back.txt
 create mode 100644 documents/tareas.txt

D:\repoSMA\repopruebas>git push
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/marquinius04/repopruebas.git
 07731ef..1066582  main -> main
```

Figura 40. Resultado de la ejecución de comandos.

8. Ejemplo práctico con características

Para afianzar el uso de Git y evitar que olvides subir tus cambios, el flujo diario para crear, integrar y publicar cualquier nueva funcionalidad (por ejemplo, una característica de prueba llamada 543) se realiza con esta secuencia de comandos exacta:

8.1. Creación y salto a la nueva funcionalidad

Asegúrate de estar situado en develop y crea tu rama local de trabajo:

```
git checkout develop
git branch 543
git checkout 543
```

8.2. Modificación de archivos e indexación

Edita los archivos que necesites en VS Code y comprueba cómo cambian de estado. Cuando termines, prepara y registra los cambios de forma local:

```
git add .
git commit -m "ac 543"
```

8.3. Fusión en la rama de integración local

Vuelve a develop y unifica tu característica:

```
git checkout develop
git merge 543
```

8.4. Sincronización remota de todas las ramas afectadas

Lo que haces en tu repositorio local no se refleja automáticamente en el servidor remoto. Si quieres que GitHub muestre el mismo estado de ramas y archivos unificados que tienes en local, debes hacer push de forma explícita en cada una de las ramas implicadas, cambiando a 543 y haciendo git push, así con todas las que quieras que aparezcan en el repositorio remoto.

9. Estados de ramas y ejemplos

9.1. Ramas alineadas

Al inicio, tanto la rama develop como la rama master o main están completamente fusionadas y contienen la misma información.

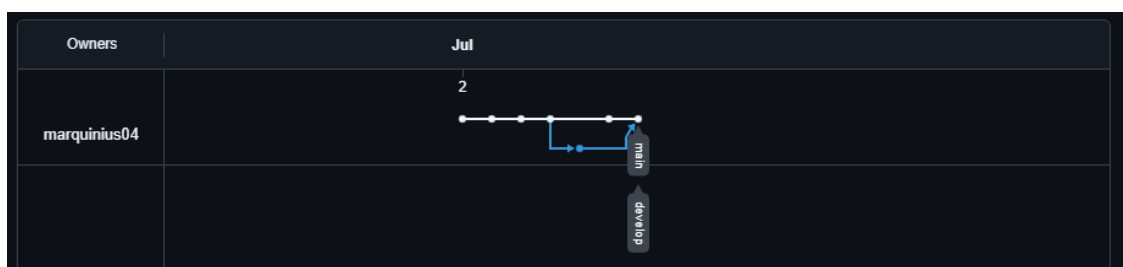


Figura 41. Vista de las ramas desde Git Graph.

9.2. Creación de la rama otra y primer desvío

Crearemos una rama nueva llamada “otra” para comprobar los cambios en el árbol:

1. Cambiar a la rama develop
2. Crear la rama “otra”
3. Generar cambios locales: Se crea un nuevo archivo llamado “nuevaotra.txt” en el editor
4. Realizamos el commit
5. Subimos la rama al servidor remoto por primera vez utilizando:

```
git push --set-upstream origin otra
```

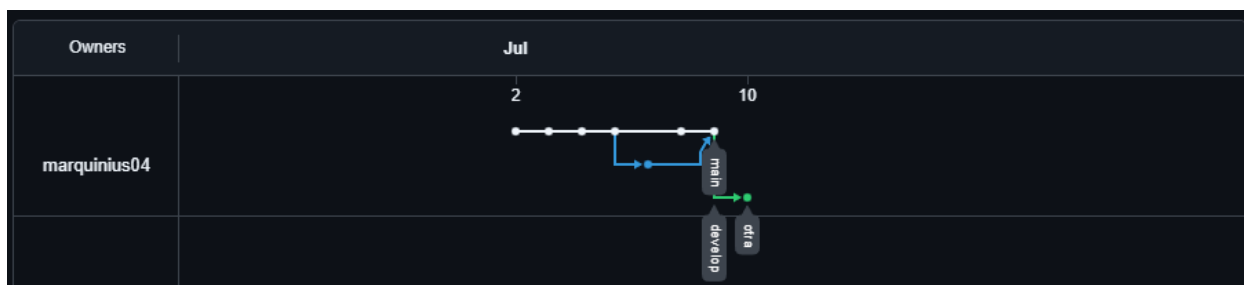


Figura 42. Creación de la rama “otra” que se desalinea de “develop”.

Podemos observar que ha aparecido una bifurcación nueva, con la creación de la nueva rama.

9.3. Integración en desarrollo

Ahora tenemos que integrar la rama de otra a la rama develop, y veremos como cambia el gráfico:

1. Cambiamos a develop mediante:

```
git checkout develop
```

2. Fusionamos la característica utilizando:

```
git merge otra
```

3. Subimos los cambios del merge con:

```
git push
```

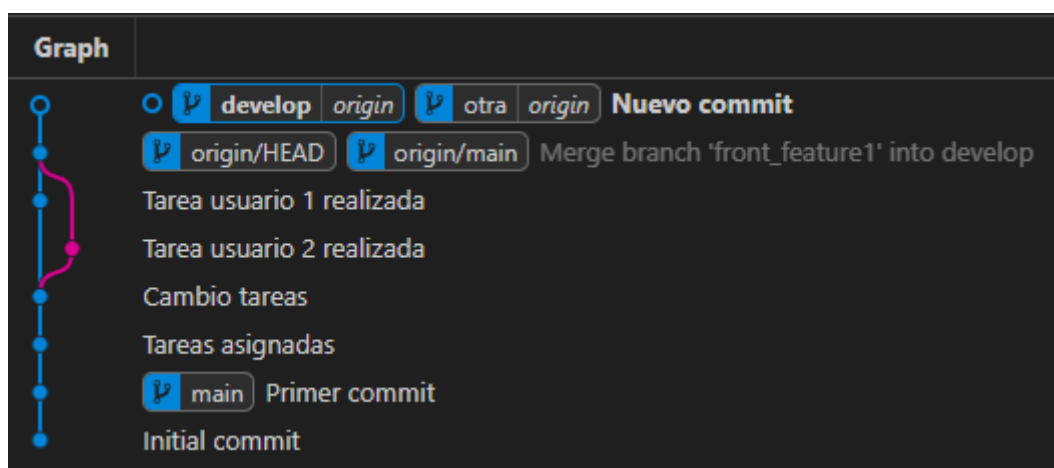


Figura 42. Resultado de merge con develop.

Si, por lo que sea, el gráfico de GitHub no se actualiza, utilizamos el de git graph. Podemos observar que la rama develop ya está al día con la nueva rama “otra”, que le hemos mergeado.

9.4. Despliegue a producción

Vamos a subir los cambios de develop a la rama principal para su despliegue:

1. Cambiamos a la rama de principal:

```
git checkout master
```

o

```
git checkout main
```

2. Actualizamos la rama principal con los cambios de develop con:

```
git merge develop
```

3. Subimos los cambios con un:

```
git push
```

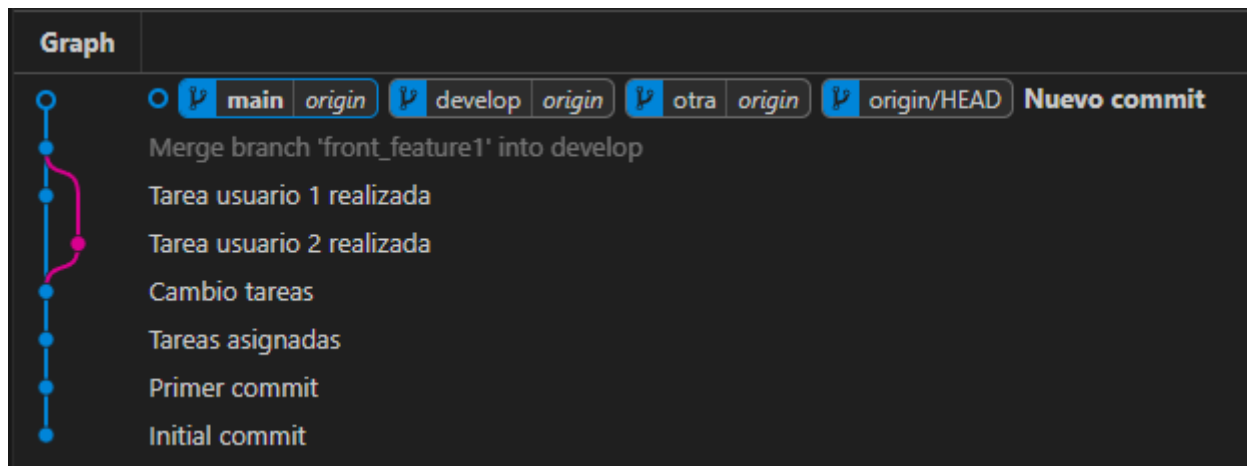


Figura 43. Despliegue en la rama principal.

Podemos observar que ahora main aparece al lado de develop y otra. Esto significa que las tres están ya al día entre sí.

9.5. Limpieza de la rama temporal

Como el trabajo en la rama otra ya está terminado, procederemos a eliminar la rama para mantener limpio el repositorio:

1. Comprobamos las ramas existentes con:

```
git branch -a
```

2. Eliminamos la rama localmente con:

```
git branch -d otra
```

3. Eliminamos la rama en el servidor de GitHub con:

```
git push origin :otra
```

```
D:\repoSMA\repopruebas2>git branch -a
develop
* main
otra
remotes/origin/HEAD -> origin/main
remotes/origin/develop
remotes/origin/main
remotes/origin/otra

D:\repoSMA\repopruebas2>git branch -d otra
Deleted branch otra (was 98d5629).

D:\repoSMA\repopruebas2>git push origin :otra
To https://github.com/marquinius04/repopruebas.git
- [deleted]          otra
```

Figura 44. Resultado de la limpieza de ramas.

Esto es lo que tendría que salir, a continuación comprobamos el gráfico de las ramas, ya sea con la extensión de VS Code “Git Graph”, o la pestaña Network que hay dentro de Insights en nuestro repositorio de GitHub:

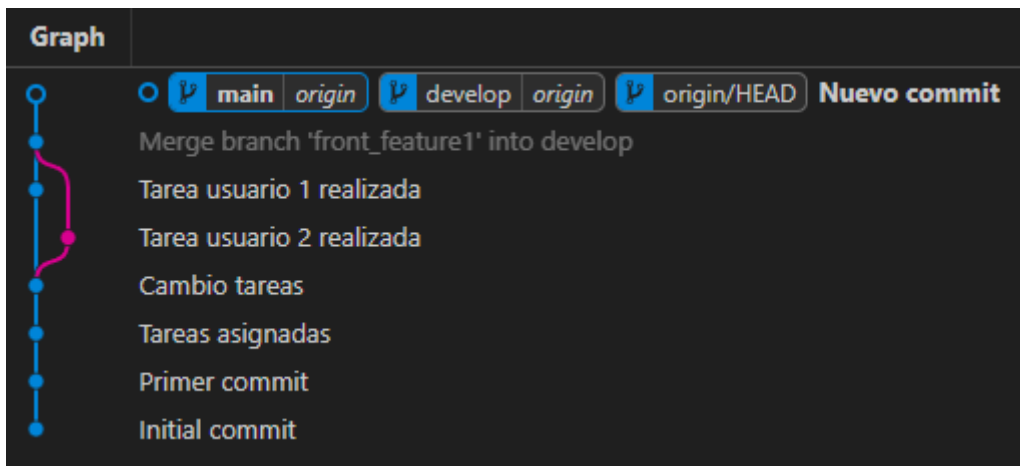


Figura 45. Estado de las ramas tras la limpieza.

En este gráfico observamos que hemos eliminado la rama “otra” correctamente, y que ahora develop y main están en la misma versión con los cambios de “otra” incluidos.

9.6. Ramas paralelas, integración parcial y resolución de conflictos

9.6.1. Configuración de la primera característica

Comenzaremos creando una rama de trabajo a partir del estado actual de desarrollo.

1. Cambiamos a la rama de develop:

```
git checkout develop
```

2. Creamos una nueva rama y nos cambiamos a esta con:

```
git branch prueba
git checkout prueba
```

3. Modificamos cualquier archivo, por ejemplo, en nuevaotra.txt cambiamos “otra” por “prueba”

4. Guardamos los cambios del archivo y realizamos el commit con:

```
git add .
git commit -m "Cambios prueba"
```

5. Subimos el commit con:

```
git push --set-upstream origin prueba
```

9.6.2. Trabajo en paralelo con la segunda característica

Para simular el trabajo de otro desarrollador en paralelo, creamos una segunda rama partiendo exactamente del mismo punto de desarrollo original.

1. Nos situamos en la rama de develop con:

```
git checkout develop
```

2. Creamos una nueva rama, llamada prueba2. Luego accedemos a ella:

```
git branch prueba2
```

```
git checkout prueba2
```

3. Realizamos cambios en otro archivo distinto al que tocamos en la rama “prueba”, por ejemplo, creamos uno nuevo llamado “nuevapueba.txt” y escribimos cualquier texto en él.

4. Guardamos el archivo y realizamos el commit:

```
git add .  
git commit -m "Nueva rama prueba2"
```

5. Subimos la rama al repositorio remoto con:

```
git push --set-upstream origin prueba2
```

9.6.3. Fusión y limpieza de la segunda característica

Una vez finalizada la tarea de la segunda rama, la fusionamos en develop y la eliminamos para mantener limpio el repositorio.

1. Nos situamos en la rama develop con:

```
git checkout develop
```

2. Fusionamos los cambios de prueba2:

```
git merge prueba2
```

3. Subimos el merge de prueba2 con develop a GitHub:

```
git push
```

4. Eliminamos la rama prueba2 de forma segura:

```
git branch -d prueba2
```

5. Borramos la rama del repositorio remoto con:

```
git push origin :prueba2
```

9.6.4. Despliegue anticipado a producción

En ocasiones, es necesario desplegar el estado actual de develop a producción (master o main) aunque todavía existan ramas de características (prueba) abiertas y sin fusionar en el proyecto.

1. Nos situamos en la rama de producción con:

```
git checkout master
```

o

```
git checkout main
```

2. Fusionamos los cambios de develop a producción:

```
git merge develop
```

3. Subimos el merge de develop con master a GitHub:

```
git push
```

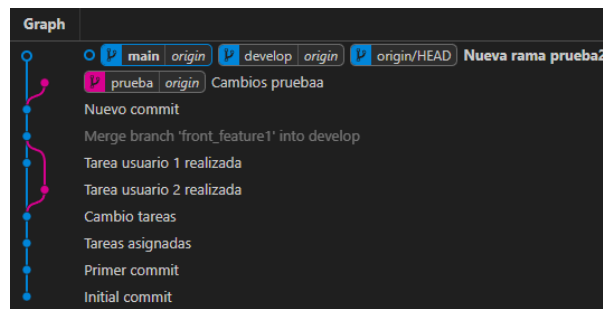


Figura 46. Resultado de la ejecución de los comandos.

Este tipo de flujos debe vigilarse de cerca. En el gráfico de GitHub se observará que la rama prueba se ha quedado desactualizada y rezagada en el historial por detrás del punto de producción.

9.6.5. Continuación del desarrollo en paralelo (prueba3)

Mientras la rama prueba sigue abierta, el equipo continúa trabajando en desarrollo y crea una nueva funcionalidad (prueba 3) a partir del nuevo estado del proyecto.

1. Nos situamos en la rama de develop de nuevo:

```
git checkout develop
```

2. Creamos y nos situamos en la rama prueba3:

```
git branch prueba3
git checkout prueba3
```

3. Modificamos el archivo “nuevaotra.txt” y, tras borrar todo, escribimos “ya estoy en prueba3” dentro de él.

4. Guardamos los cambios del archivo, realizamos el commit y lo subimos:

```
git add .
git commit -m "Nueva rama prueba3"
git push
```

5. Fusionamos prueba3 en develop:

```
git checkout develop
git merge prueba3
git push
```

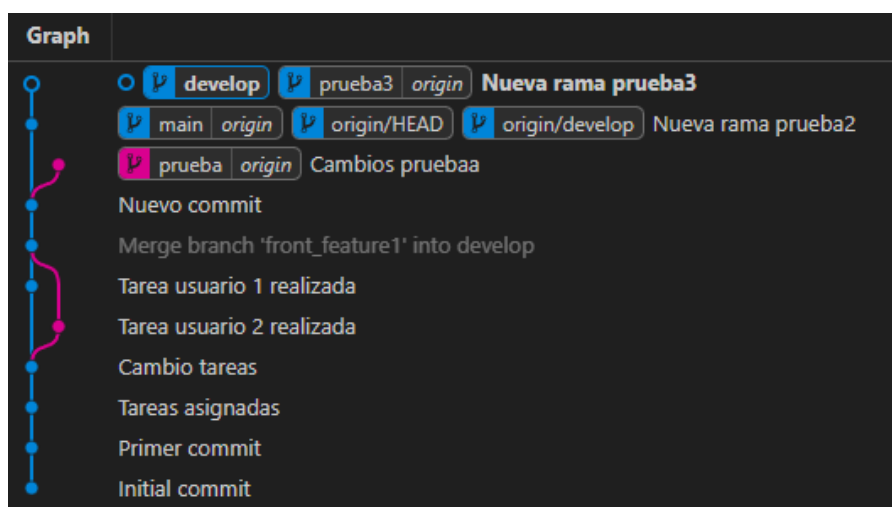


Figura 47. Resultado de la ejecución de los comandos.

Tras haber mergeado develop a la rama de producción, hemos creado la nueva rama prueba3 y, tras hacer cambios nuevos en esta, la hemos mergeado de nuevo a develop, por tanto ambas están una versión por delante de producción, y prueba se ha quedado atrás.

9.6.6. Finalización y subida de la rama rezagada

Ahora, retomamos la rama de prueba, que se había quedado atrasada en el desarrollo del proyecto, y la queremos subir al repositorio:

1. Cambiamos a la rama prueba:

```
git checkout prueba
```

2. Editamos el archivo "nuevaotra.txt" y escribimos cualquier cosa.

3. Guardamos los cambios del archivo, realizamos el commit y lo subimos:

```
git add .  
git commit -m "Fin de prueba"  
git push
```

9.6.7. Fusión de la rama conflictiva en develop y resolución en VS Code

1. Cambiamos a la rama develop:

```
git checkout develop
```

2. Mergeamos la rama prueba y nos saldrá un conflicto:

```
git merge prueba
```

```
D:\repoSMA\repopruebas2>git merge prueba  
Auto-merging nuevaotra.txt  
CONFLICT (content): Merge conflict in nuevaotra.txt  
Automatic merge failed; fix conflicts and then commit the result.
```

Figura 48. Resultado de la ejecución de los comandos.

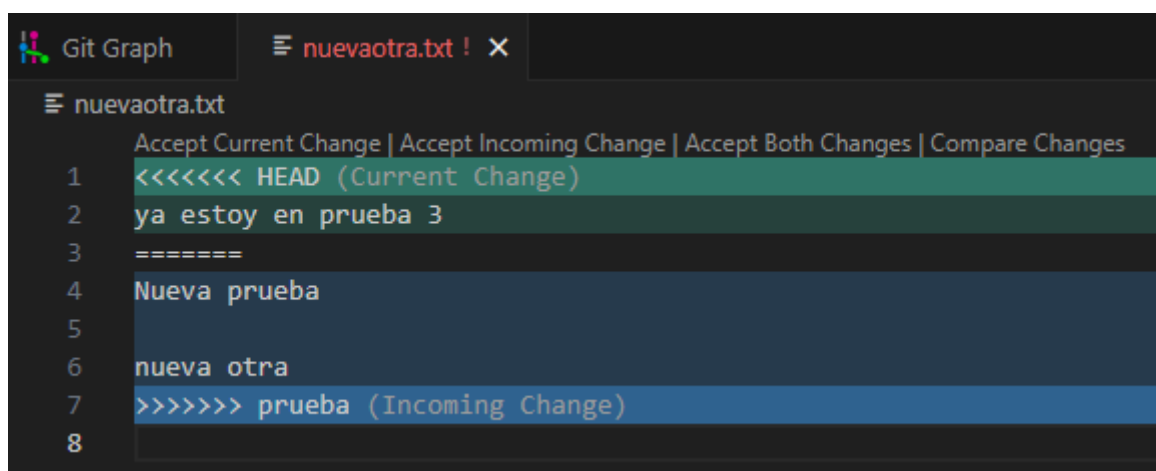


Figura 49. Vista desde VS Studio.

3. Este conflicto se debe a que hemos tocado el mismo archivo en ambas ramas, y al mergear, nos pregunta que con cuál de las dos versiones nos queremos quedar. Seleccionaremos que nos queremos quedar con ambos cambios y retocamos el archivo de esta forma.

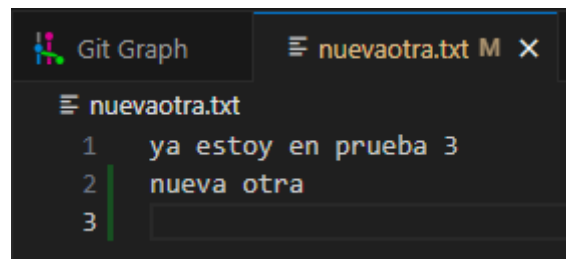


Figura 50. Resultado de resolver el conflicto.

4. Luego guardamos los cambios y los subimos al repositorio:

```
git add .
git commit -m "Merge branch 'prueba' into develop"
git push
```

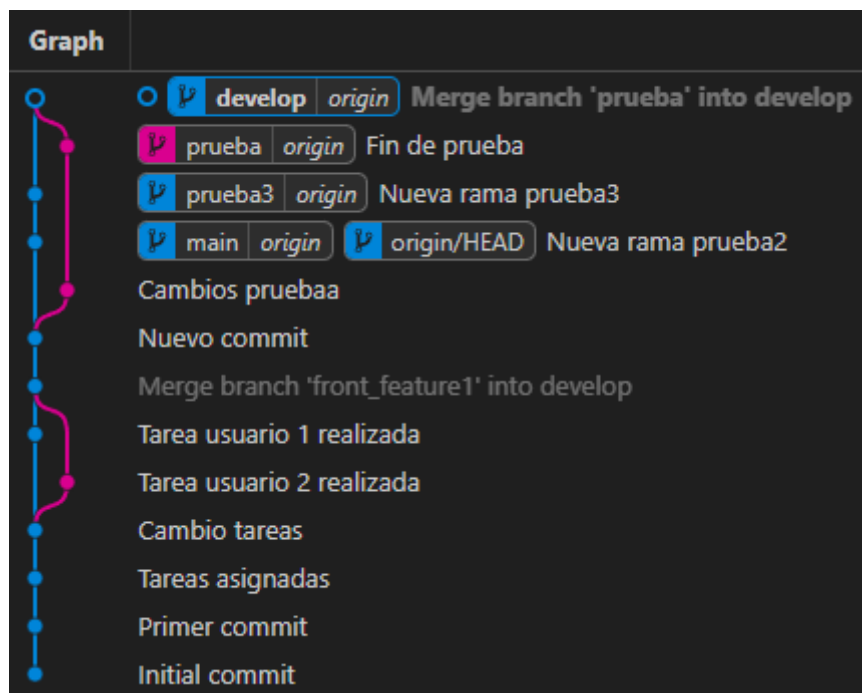


Figura 51. Resultado al guardar cambios.

Como resultado de haber resuelto un conflicto y modificado archivos directamente en develop durante la fusión de prueba, la rama prueba 3 ya no es exactamente igual a la rama develop actual. Si el desarrollador de la rama prueba 3 quisiera seguir trabajando en ella, tendría que traerse los cambios del servidor. Este proceso de actualización se detallará en el siguiente capítulo de la guía.

9.6.8. Sincronizar ramas

La rama develop contiene ahora commits y código que la rama prueba3 no tiene. Lo primero es traernos esos cambios de vuelta a nuestra rama de trabajo.

1. Cambiamos a la rama de característica con:

```
git checkout prueba3
```

2. Traemos los cambios de develop a la rama prueba3:

```
git merge develop
```

3. Subimos los cambios traídos desde develop a la rama prueba3:

git push

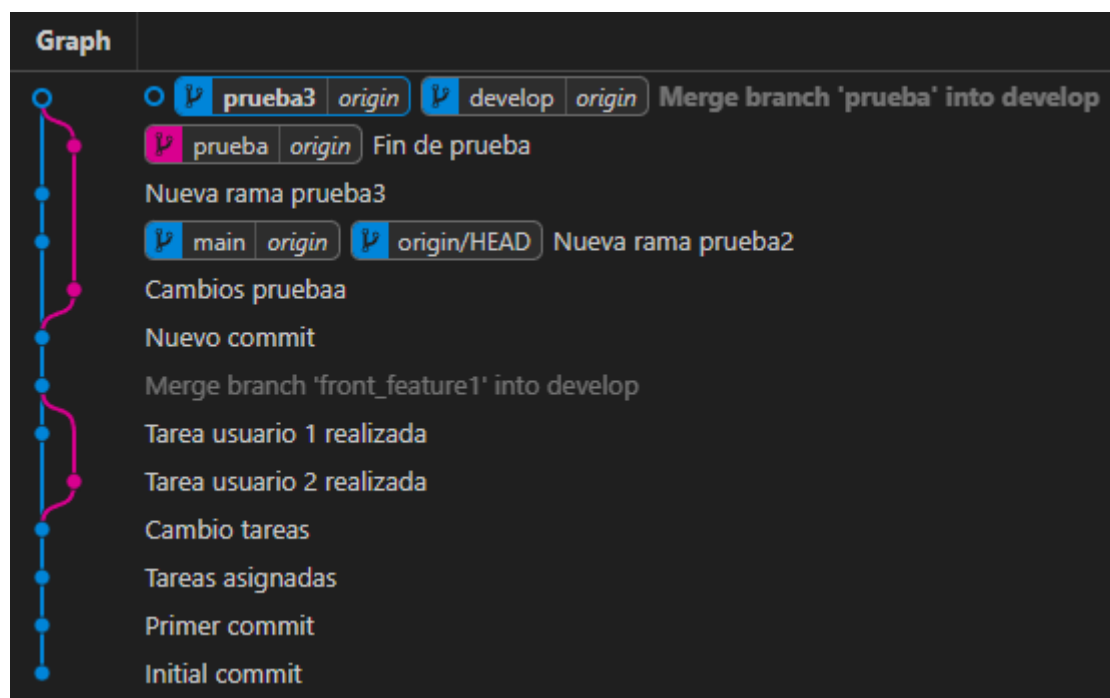


Figura 52. Resultado tras el push.

Podemos observar que prueba3 y develop ya están al día, tras realizar la sincronización de ambas. Ahora realizaremos exactamente los mismos pasos para la rama prueba, que también había quedado desactualizada en relación a la rama develop.

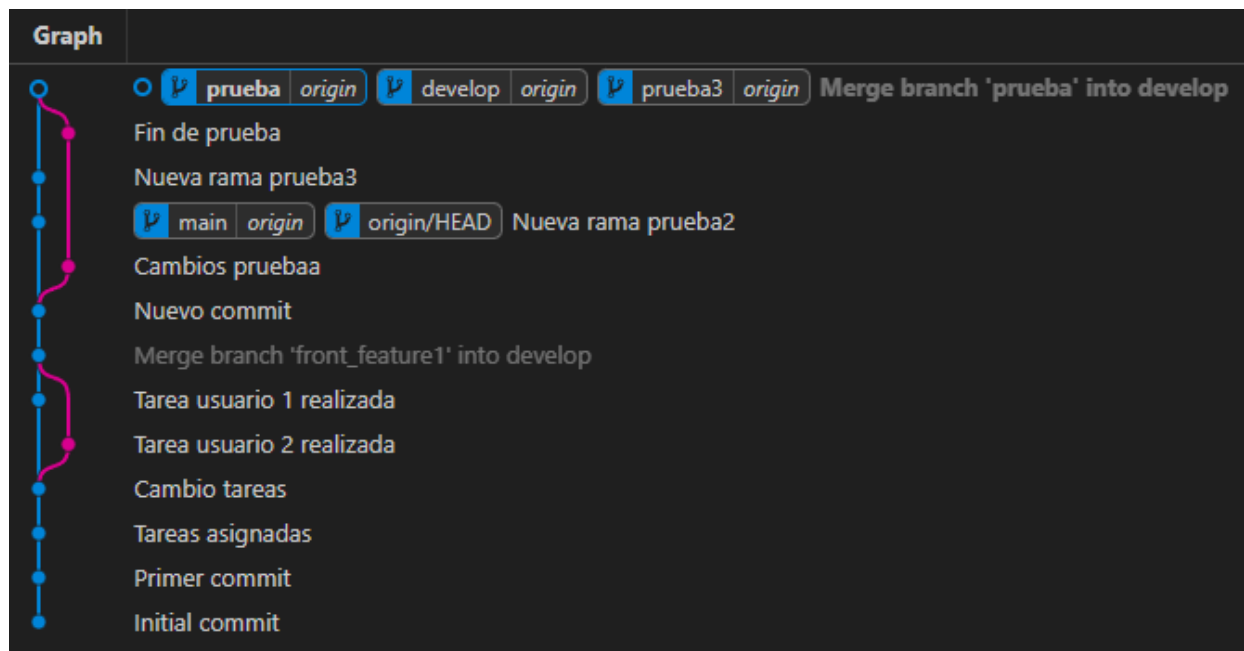


Figura 52. Resultado final.

Esto es lo que debería de quedar tras sincronizar los cambios de develop con las ramas prueba3 y prueba.

9.6.9. Limpieza de ramas de características

Tras finalizar la sincronización de ambas ramas, lo que tenemos que hacer es, como con todas las ramas de

características finalizadas e implementadas en develop, borrarlas del repositorio local y del repositorio en línea. Para ello hacemos:

1. Primero nos cambiamos a una rama que no vayamos a borrar, como la principal o develop con git checkout.
2. Después usamos los siguientes comandos para borrarlas de nuestro repositorio local:

```
git branch -d prueba
git branch -d prueba3
```

3. Eliminamos nuestras ramas de características de repositorio de GitHub con:

```
git push origin :prueba
git push origin :prueba3
```

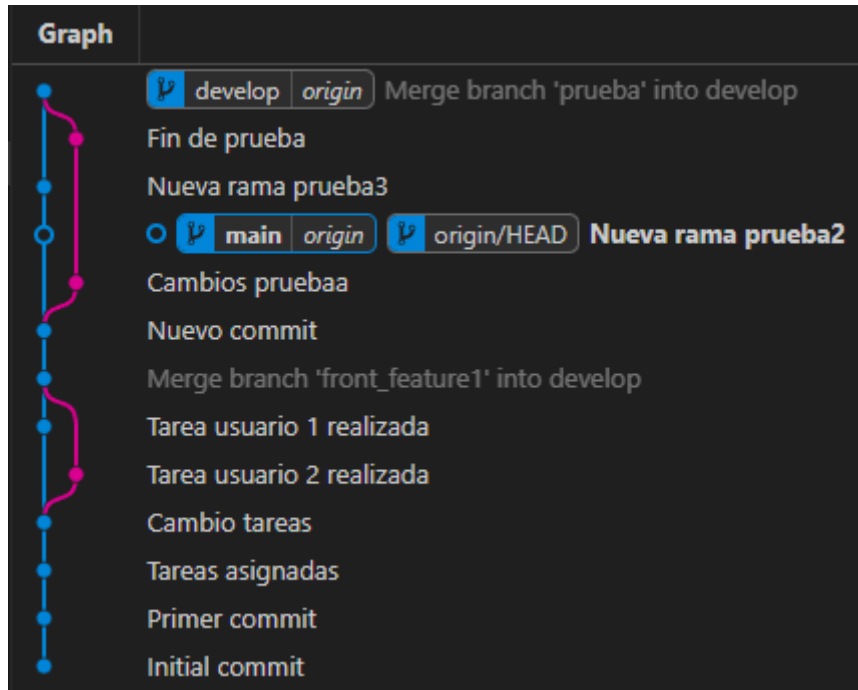


Figura 53. Resultado tras eliminar las ramas.

Así es como debería de quedar el gráfico tras la eliminación de las ramas prueba3 y prueba. En él podemos observar que develop está por delante de main en cuanto a cambios, por lo que hay que poner la rama principal al día, esto lo haremos con un merge.

9.6.10. Despliegue a producción

Después de realizar la limpieza de ramas de características, hay que realizar el despliegue a producción, es decir, poner a master (o main) al día con develop, de forma que queden las dos en la misma versión. Esto lo hacemos así:

1. Cambiamos a la rama principal:

```
git checkout main
```

2. Fusionamos los cambios de develop a la rama principal:

```
git merge develop
```

3. Subimos los cambios con:

```
git push
```

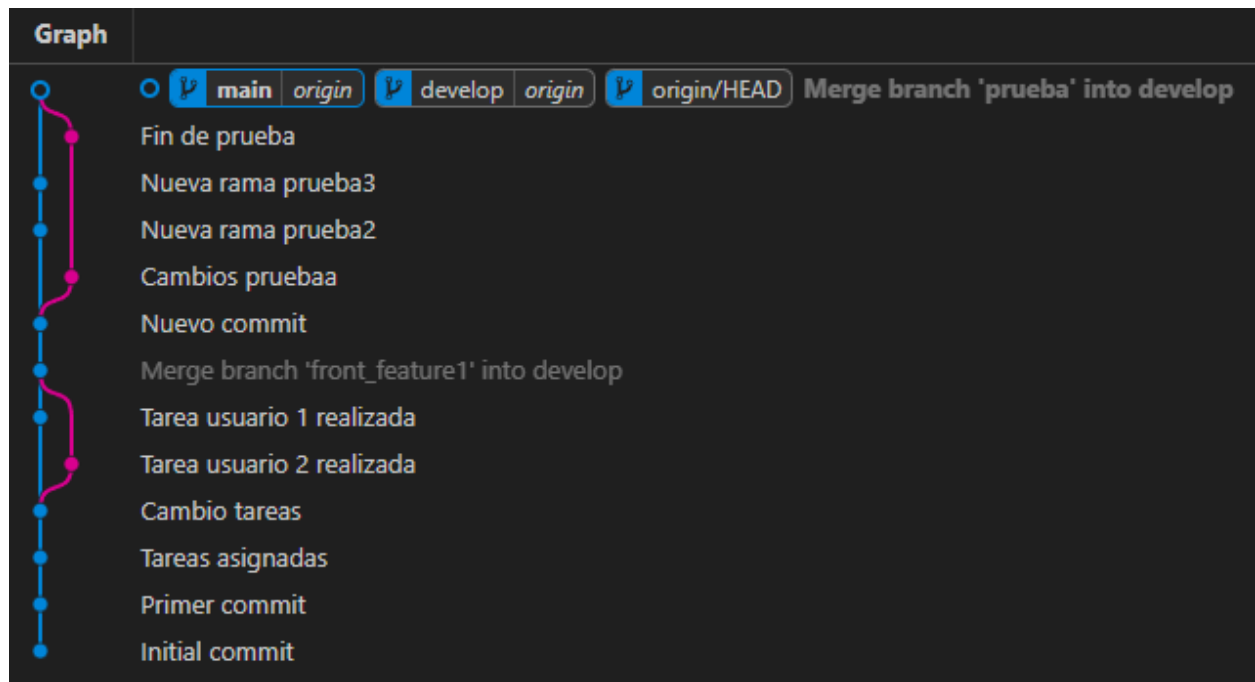


Figura 54. Resultado del despliegue.

Y así es como debería de quedar el gráfico tras añadir todas las características desarrolladas en ramas de características a develop, y fusionar develop a la rama principal.